

# 密码杂凑算法 Litchi 算法文本

设计者：贾文义<sup>1,2</sup>，刘玉鹏<sup>1</sup>，白亮<sup>1,2</sup>，胡红钢<sup>3</sup>，  
李昌伦<sup>1,2</sup>，刘益同<sup>1</sup>，彭烨<sup>1</sup>，吴嘉力<sup>1</sup>

完成单位：<sup>1</sup>兴唐通信科技有限公司，<sup>2</sup>抗量子计算密码北京市重点实验室，<sup>3</sup>中国科学技术大学

日期：2026 年 6 月 29 日



## 目 录

|                           |    |
|---------------------------|----|
| 1 引言 .....                | 1  |
| 2 算法描述 .....              | 1  |
| 2.1 简介 .....              | 1  |
| 2.1.1 符号说明 .....          | 1  |
| 2.1.2 字序和约定 .....         | 2  |
| 2.2 算法上层结构 .....          | 3  |
| 2.2.1 Litchi 算法.....      | 3  |
| 2.2.2 消息填充 .....          | 4  |
| 2.3 置换 P .....            | 4  |
| 2.3.1 P 状态 .....          | 5  |
| 2.3.2 常数加变换 $\iota$ ..... | 5  |
| 2.3.3 非线性变换 $\chi$ .....  | 6  |
| 2.3.4 线性变换 $\theta$ ..... | 7  |
| 2.3.5 换位变换 $\pi$ .....    | 8  |
| 2.4 HASH 函数 .....         | 8  |
| 2.5 XOF 函数 .....          | 8  |
| 3 设计原理 .....              | 8  |
| 3.1 上层结构设计 .....          | 8  |
| 3.2 置换 P 设计 .....         | 10 |
| 3.2.1 非线性变换设计 .....       | 10 |
| 3.2.2 线性变换设计 .....        | 12 |
| 3.2.3 换位变换设计 .....        | 13 |
| 3.2.4 常数加设计 .....         | 14 |
| 3.3 参数选择依据 .....          | 15 |
| 4 安全性声明与分析 .....          | 15 |

|                           |    |
|---------------------------|----|
| 4.1 理论安全性 .....           | 15 |
| 4.1.1 上层结构安全性分析 .....     | 15 |
| 4.1.2 置换 P 安全性分析 .....    | 18 |
| 4.2 具体安全性 .....           | 27 |
| 4.3 其他安全特性 .....          | 28 |
| 4.3.1 统计随机性测试 .....       | 28 |
| 4.3.2 抗长度扩展攻击安全性分析 .....  | 31 |
| 5 性能评估 .....              | 31 |
| 5.1 性能分析 .....            | 31 |
| 5.1.1 整体结构的设计效率 .....     | 31 |
| 5.1.2 核心组件的软硬件效率 .....    | 32 |
| 5.2 性能测试 .....            | 32 |
| 5.2.1 软件性能测试 .....        | 32 |
| 5.2.2 硬件性能测试 .....        | 34 |
| 6 特点声明 .....              | 36 |
| 6.1 算法的创新性 .....          | 36 |
| 6.2 算法的简洁性 .....          | 37 |
| 6.3 算法的灵活性 .....          | 37 |
| 6.4 算法在不同平台上的实现性能优势 ..... | 37 |
| 7 与 SHA-3 算法比较 .....      | 38 |
| 8 参考文献 .....              | 38 |

# 1 引言

为响应中国商用密码标准化研究院举办的新一代商用密码杂凑算法征集活动，我们研制完成了密码杂凑算法 Litchi，共包含 3 个输出长度分别为 512、768 和 1024 比特的杂凑算法以及 1 个扩展输出函数。依据杂凑输出长度的不同，3 个杂凑算法分别命名为 Litchi-512、Litchi-768 和 Litchi-1024，扩展输出函数命名为 Litchi-XOF。4 个算法的具体特征见表 1-1。

表 1-1 Litchi 具体特征

| 算法名称        | 输入消息比特长度        | 输出杂凑比特长度        |
|-------------|-----------------|-----------------|
| Litchi-512  | $\leq 2^{64}-1$ | 512             |
| Litchi-768  | $\leq 2^{64}-1$ | 768             |
| Litchi-1024 | $\leq 2^{64}-1$ | 1024            |
| Litchi-XOF  | $\leq 2^{64}-1$ | $\leq 2^{64}-1$ |

Litchi 杂凑算法能够抵抗已知经典计算攻击和量子计算攻击，能够在广泛的软件与硬件平台上高效实现，具有安全性、高效性和灵活性等特点。

本文档将给出算法描述、算法的设计理念、初步的安全性分析以及实现性能评估。

## 2 算法描述

### 2.1 简介

Litchi 算法基于文献[GHJ+25]提出的 Sponge- $F^{pd}$  结构设计，内部压缩函数采用一个状态规模为 1600 比特的置换。基于 Litchi 算法，我们给出 Litchi-512、Litchi-768 和 Litchi-1024 杂凑算法，输出长度分别为 512、768 和 1024 比特，同时设计了扩展输出函数 Litchi-XOF。

#### 2.1.1 符号说明

- 0x            用前缀0x表示16进制数。
- =            赋值运算符，也可表示相等。
- +            算术加法运算符。

|                   |                                    |
|-------------------|------------------------------------|
| $\oplus$          | 逐比特异或运算符。                          |
| $\&$              | 逐比特与运算符。                           |
| $\text{mod}$      | 整数取余运算符。                           |
| $\parallel$       | 字符串连接运算符。                          |
| $\#$              | 集合中元素的数量。                          |
| $\min$            | 从若干个元素中取最小值。                       |
| $\lll n$          | 操作数循环左移 $n$ 比特运算符。                 |
| $\ggg n$          | 操作数循环右移 $n$ 比特运算符。                 |
| $\ll n$           | 操作数左移 $n$ 比特运算符，低位补0。              |
| $\gg n$           | 操作数右移 $n$ 比特运算符，高位补0。              |
| $\{0, 1\}^n$      | $n$ 比特串空间。                         |
| $A^T$             | 矩阵（或向量） $A$ 的转置操作。                 |
| $\text{MSB}_n(x)$ | $x$ 的最高 $n$ 比特数。                   |
| $\text{LSB}_n(x)$ | $x$ 的最低 $n$ 比特数。                   |
| $0^n$             | $n$ 比特的全0串。                        |
| $[i]_n$           | 小于 $2^n$ 的非负整数 $i$ 对应的 $n$ 比特二进制串。 |
| $\lceil \rceil$   | 向上取整运算符。                           |
| $N$               | 置换 $P$ 的迭代轮数，设置为16。                |
| $\text{RC}_i$     | 第 $i$ 轮的64比特轮常数。                   |

### 2.1.2 字序和约定

约定数据变量用最高比特（或分块）在左边，最低比特（或分块）在

右边表示，从左到右从 0 开始编号。例如，一个  $n$  比特数从高到低（从左到右）分别称为它的第 0 比特、第 1 比特、第 2 比特、...、第  $n-1$  比特。当一个变量被分成几个分块时，最左边（最高）分块由原始数据的最高部分组成，最右边（最低）分块由原始数据的最低部分组成。

约定数据采用大端序，当输入（或输出）是字或字节流时，地址小的是高位数据（即左边），地址大的是低位数据（即右边）。每 8（或 4）个字节组成一个 64（或 32）比特字。

除非特别说明，下文所有的多项式、向量和矩阵都定义在二元域中。

## 2.2 算法上层结构

Litchi 算法上层结构由消息填充、消息压缩和杂凑输出三个阶段组成。

消息填充阶段，根据安全等级参数  $c$ ，对输入的消息  $M$  进行填充，使之满足规定的长度要求，得到  $l$  块等长的消息块  $M_{\text{pad}}=M_0||M_1||\cdots||M_{l-1}$ 。

消息压缩阶段，从初始状态  $S_0$  开始，对填充后的消息块  $M_i$  进行迭代压缩，更新杂凑状态  $S_i$ 。

杂凑输出阶段，以消息压缩阶段产生的杂凑状态为输入，根据输出长度  $|h|$  进行扩展输出，得到杂凑值  $h=\text{MSB}_{|h|}(h_1||h_2||\dots||h_{\lceil |h|/c \rceil})$ 。

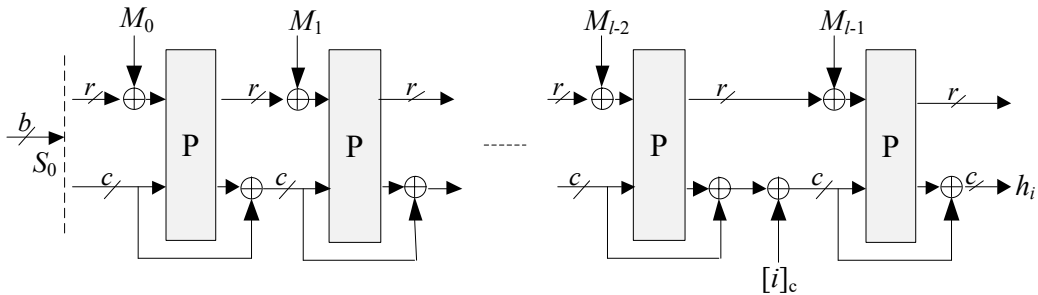


图 2-1 Litchi 算法上层结构图

Litchi 上层结构如图 2-1 所示，其中  $P$  是  $\{0, 1\}^b$  到  $\{0, 1\}^b$  的置换， $b=r+c$ ， $M_i(0 \leq i \leq l-1)$  长度为  $r$  比特， $h_i(1 \leq i \leq \lceil |h|/c \rceil)$  长度为  $c$  比特。与  $\text{Sponge-F}^{\text{P,pd}}$  结构不同的是，在可变杂凑长度输出时，Litchi 上层结构采用基于计数器的并行方式，而非串行迭代方式。

### 2.2.1 Litchi 算法

对于 Litchi 算法， $b=1600$ 。为了达到支持不同安全等级和不同功能输

出的目的，我们使用安全参数  $c$  和功能标识  $fid$  对 Litchi 进行控制， $fid=0$  时用于杂凑算法， $fid=1$  时用于扩展输出函数。

输入消息  $M$  的比特长度为  $|M|$ ，要求  $|M| \leq 2^{64}-1$ ；输出杂凑值  $h$  的比特长度为  $|h|$ ，当  $fid=0$  时，要求  $|h| \leq c-64$ ，当  $fid=1$  时，要求  $|h| \leq 2^{64}-1$ 。

$h = \text{Litchi}[c, fid](M, |h|)$  具体计算如下：

(1) 计算  $M_0 || M_1 || \dots || M_{l-1} = \text{Pad}[c](M)$ ；

(2)  $S_0 = [2^{63}fid + c]_{1600}$ ；

(3) 对于  $1 \leq i \leq l-1$ ，执行：

$$S_i = P(S_{i-1} \oplus (M_{i-1} || 0^c)) \oplus (0^r || \text{LSB}_c(S_{i-1}));$$

(4) 对于  $1 \leq i \leq \lceil |h|/c \rceil$ ，执行：

$$1) ST = S_{l-1} \oplus (M_{l-1} || [i]_c);$$

$$2) h_i = \text{LSB}_c(P(ST)) \oplus \text{LSB}_c(ST);$$

(5) 返回  $h = \text{MSB}_{|h|}(h_1 || h_2 || \dots || h_{\lceil |h|/c \rceil})$ 。

### 2.2.2 消息填充

$\text{Pad}$  为安全参数  $c$  控制下的消息填充函数，填充方式为首先将消息  $M$  添加 1 比特 1，然后添加最少比特的 0，使得填充后消息  $M_{\text{pad}}$  长度为  $1600-c$  比特的整数倍。 $M_{\text{pad}} = \text{Pad}[c](M)$  具体计算如下：

$$M_{\text{pad}} = M_0 || M_1 || \dots || M_{l-1} = M || 1 || 0^{r-1-(|M| \bmod r)}.$$

其中， $|M|$  为消息  $M$  的比特长度， $r=1600-c$ ， $l = \lceil (|M|+1)/r \rceil$ ， $M_i (0 \leq i \leq l-1)$  的长度为  $r$  比特。

### 2.3 置换 P

Litchi 杂凑密码算法使用了一个  $\{0,1\}^{1600}$  到  $\{0,1\}^{1600}$  的置换  $P$ ，置换  $P$  使用了 SPN 结构，由轮函数  $f$  迭代  $N$  轮而成。置换  $P$  的输入为一个 1600 比特的状态  $A$ ，输出是一个 1600 比特的状态  $B$ ， $B=P(A)$ ，执行过程如下：

(1)  $A_0 = A$ ；

(2) 对于  $i = 0$  到  $N-1$ ，执行：

$$A_{i+1} = f(A_i, i_r);$$

(3) 返回  $B = A_N$ 。

其中，轮函数  $f$  的输入为一个 1600 比特的状态  $A_i$  和一个轮数标识  $i_r$ ， $f$  由 4 部分组成，分别是常数加变换  $\mathfrak{t}$ 、非线性变换  $\chi$ 、线性变换  $\theta$  和换位变换  $\pi$ 。具体为： $f(A_i, i_r) = \pi(\theta(\chi(\mathfrak{t}(A_i, i_r))))$ 。

### 2.3.1 P 状态

1600 比特置换 P 的状态 (state) 由 5 个 320 比特的面 (plane) 组成，每个面由 5 个 64 比特的棱 (lane) 组成，即 P 的状态可看作一个  $5 \times 5 \times 64$  的立方体。

A 表示整个状态，为 1600 比特； $A[i]$  表示 A 的第  $i$  个面，为 320 比特， $0 \leq i \leq 4$ ； $A[i][j]$  表示  $A[i]$  的第  $j$  个棱，为 64 比特， $0 \leq j \leq 4$ ； $A[i][j][k]$  表示  $A[i][j]$  的第  $k$  比特，为 1 比特， $0 \leq k \leq 63$ 。

$$A = A[0] \parallel A[1] \parallel \cdots \parallel A[4];$$

$$A[i] = A[i][0] \parallel A[i][1] \parallel \cdots \parallel A[i][4], 0 \leq i \leq 4;$$

$$A[i][j] = A[i][j][0] \parallel A[i][j][1] \parallel \cdots \parallel A[i][j][63], 0 \leq i \leq 4, 0 \leq j \leq 4.$$

### 2.3.2 常数加变换 $\mathfrak{t}$

常数加变换  $\mathfrak{t}$  是将第  $r$  轮的 64 比特轮常数  $RC_r$  加到棱  $A[0][0]$  中，其余棱保持不变， $B = \mathfrak{t}(A, r)$  的具体操作如下：

(1)  $B = A$ ;

(2)  $B[0][0] = A[0][0] \oplus RC_r$ ;

(3) 返回 B。

$N$  个 64 比特长的轮常数  $RC_r$  ( $0 \leq r \leq N-1$ ) 通过常数 C 计算得到，具体值见表 2-1。常数 C 为自然常数  $e$  小数部分的前 64 比特，用 16 进制表示为：

$$C = 0xb7e151628aed2a6a。$$

$RC_r = 2^r \otimes C$ ，“ $\otimes$ ”为有限域  $GF(2^{64}) = GF(2)[x]/(x^{64} + x^{63} + x^{62} + x^{54} + x^{40} + x^{35} + x^{15} + x + 1)$  中的乘法。

表 2-1 轮常数  $RC_r$ 

| 轮数 $r$ | 轮常数 $RC_r$         | 轮数 $r$ | 轮常数 $RC_r$         |
|--------|--------------------|--------|--------------------|
| 0      | 0xb7e151628aed2a6a | 8      | 0x5dd1971aed536b16 |
| 1      | 0xaf82a3cd15dad4d7 | 9      | 0xbba32e35daa6d62c |
| 2      | 0x9f4546922bb529ad | 10     | 0xb7065d63b54d2c5b |
| 3      | 0xfeca8c2c576ad359 | 11     | 0xae4cbbcf6a9ad8b5 |
| 4      | 0x3dd51950aed526b1 | 12     | 0x9cd97696d5353169 |
| 5      | 0x7baa32a15daa4d62 | 13     | 0xf9f2ec25aa6ae2d1 |
| 6      | 0xf7546542bb549ac4 | 14     | 0x33a5d94354d545a1 |
| 7      | 0x2ee8cb8d76a9b58b | 15     | 0x674bb286a9aa8b42 |

### 2.3.3 非线性变换 $\chi$

非线性变换  $\chi$  是将状态 A 经过 320 个相同的 5 比特 S 盒并行更新，得到状态 B，S 盒的查找表如表 2-2，电路实现见图 2-2。

表 2-2 S 盒查找表

|      |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| x    | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 |
| S(x) | 0  | 2  | 4  | 23 | 8  | 10 | 14 | 13 | 16 | 18 | 20 | 15 | 28 | 30 | 26 | 17 |
| x    | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 | 31 |
| S(x) | 1  | 19 | 5  | 6  | 9  | 27 | 31 | 12 | 25 | 11 | 29 | 22 | 21 | 7  | 3  | 24 |

非线性变换  $B=\chi(A)$  的具体操作如下：

- (1)  $B[0] = A[0]\&A[2]\&A[3] \oplus A[2]\&A[3]\&A[4] \oplus A[0]\&A[4] \oplus A[3]\&A[4] \oplus A[1]$ ;
- (2)  $B[1] = A[1]\&A[3]\&A[4] \oplus A[0]\&A[1] \oplus A[2]$ ;
- (3)  $B[2] = A[1]\&A[2] \oplus A[3]$ ;
- (4)  $B[3] = A[2]\&A[3] \oplus A[4]$ ;
- (5)  $B[4] = A[3]\&A[4] \oplus A[0]$ ;
- (6) 返回 B。

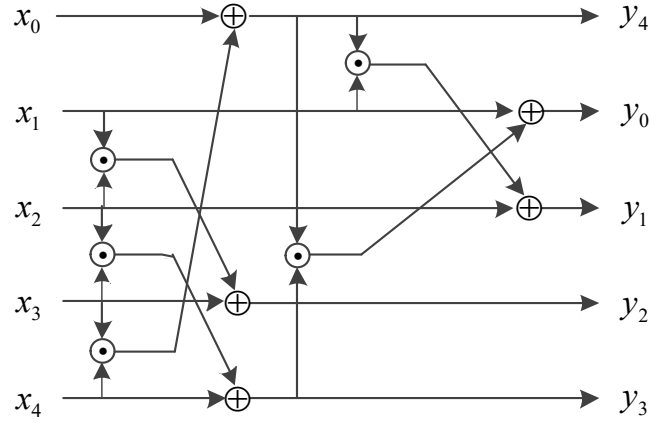


图 2-2 S 盒实现电路框图

### 2.3.4 线性变换 $\theta$

线性变换 $\theta$ 是将状态 A 的每个面  $A[i]$  分别经过线性变换 $\theta_i$  进行更新， $0 \leq i \leq 4$ 。线性变换  $B = \theta(A)$  的具体操作如下：

(1) 对于  $0 \leq i \leq 4$ ，执行：

$$B[i] = \theta_i(A[i]);$$

(2) 返回 B。

线性变换 $\theta_i$  是一个 320 比特的线性置换，由可变参数的线性置换 Theta 确定， $\theta_i$  具体如下：

$$\theta_0 = \text{Theta}[31, 1, 4, 21, 22];$$

$$\theta_1 = \text{Theta}[19, 12, 15, 34, 13];$$

$$\theta_2 = \text{Theta}[28, 35, 58, 9, 23];$$

$$\theta_3 = \text{Theta}[55, 18, 5, 27, 44];$$

$$\theta_4 = \text{Theta}[14, 43, 53, 20, 25]。$$

$\text{Theta}[t_0, t_1, t_2, t_3, t_4]$  是由 5 个参数  $(t_0, t_1, t_2, t_3, t_4)$  确定的 320 比特线性置换， $Y = \text{Theta}[t_0, t_1, t_2, t_3, t_4](X)$  操作如下：

1) 将 X 分为 5 个 64 比特块  $X_i (0 \leq i \leq 4)$ ，即  $X = X_0 || X_1 || X_2 || X_3 || X_4$ ；

2)  $E = X_0 \oplus X_1 \oplus X_2 \oplus X_3 \oplus X_4$ ；

3)  $F = (X_0 \lll t_0) \oplus (X_1 \lll t_1) \oplus (X_2 \lll t_2) \oplus (X_3 \lll t_3) \oplus (X_4 \lll t_4)$ ；

4) 对于  $0 \leq i \leq 4$ ，执行：

$$Y_i = X_i \oplus (E \ggg t_i) \oplus F;$$

5) 返回  $Y=Y_0||Y_1||Y_2||Y_3||Y_4$ 。

### 2.3.5 换位变换 $\pi$

换位变换  $\pi$  是将状态  $A$  的 25 个棱进行换位操作,  $B=\pi(A)$  的具体操作如下:

(1) 对于  $0 \leq i \leq 4, 0 \leq j \leq 4$  执行:

$$B[i][j] = A[(i+2j) \bmod 5][(3i+3j) \bmod 5];$$

(2) 返回  $B$ 。

## 2.4 HASH 函数

杂凑函数 Litchi-512、Litchi-768 和 Litchi-1024 基于 Litchi 算法构造, 输入为小于  $2^{64}$  比特的消息  $M$ , 其计算方式如下:

$$\text{Litchi-512}(M) = \text{Litchi}[576, 0](M, 512);$$

$$\text{Litchi-768}(M) = \text{Litchi}[832, 0](M, 768);$$

$$\text{Litchi-1024}(M) = \text{Litchi}[1088, 0](M, 1024)。$$

## 2.5 XOF 函数

扩展输出函数 Litchi-XOF 基于 Litchi 算法构造, 输入为长度小于  $2^{64}$  比特的消息  $M$ , 输出长度  $|xof|$  小于  $2^{64}$  比特, 其计算方式如下:

$$\text{Litchi-XOF}(M, |xof|) = \text{Litchi}[1024, 1](M, |xof|)。$$

*注: 由于设计要求没有对扩展输出函数提出具体指标, 我们这里给出一个扩展输出函数的示例, 主要考虑杂凑输出效率和安全强度, 同时兼顾 256 和 512 比特长度等常见的消息输入。实际使用时, 可根据安全强度要求以及效率的需求调整参数  $c$  的值, 给出其他版本的扩展输出函数。*

## 3 设计原理

### 3.1 上层结构设计

广泛使用的 Sponge 结构[BDPV07]由于抗(第二)原像只能达到  $c/2$  比特的安全强度[BDPV08], 当杂凑输出长度为 1024 比特时, Sponge 使用的置换  $P$  必须大于 2048 比特, 且由于考虑效率的原因, 比率  $r$  不能太小, 因此需要一个规模非常大的置换  $P$ , 这给设计和分析带来很大困难, 同时由于

容量  $c$  过大，给算法实现效率带来很大影响。

文献[GHJ+25]提出的  $\text{Sponge-F}^{\text{P,pd}}$  结构通过对  $\text{Sponge}$  结构  $c$  比特容量部分做前向反馈，增加了抵抗原像和第二原像攻击的能力。当杂凑输出长度为  $h(h \leq c)$  时， $\text{Sponge-F}^{\text{P,pd}}$  结构抗碰撞可达到  $h/2$  比特的安全强度，抗原像可达到  $h$  比特的安全强度，抗第二原像能达到  $\min(h, c - \log_2 l)$  比特的安全强度（ $l$  为第二原像查询消息的块数）。同时， $\text{Sponge-F}^{\text{P,pd}}$  结构在抗量子攻击方面具有很好的安全性。由于  $\text{Sponge-F}^{\text{P,pd}}$  结构的强抗(第二)原像能力，可降低对置换  $P$  规模的要求，比如 1600 比特的置换  $P$ ，选择  $c=1088$  时，抗原像和第二原像攻击的强度可达 1024 比特，满足 1024 比特杂凑输出长度的设计要求。

为了解决  $\text{Sponge-F}^{\text{P,pd}}$  串行迭代给效率带来的影响，结合  $\text{Sponge-F}^{\text{P,pd}}$  在最后一块消息处理时异或的非 0 常数（为抵抗消息长度扩展攻击），我们对  $\text{Sponge-F}^{\text{P,pd}}$  结构进行修改，设计了 Litchi 算法上层结构，主要修改点如下：

（1）将计数器用作常量，并行生成不同的杂凑输出  $h_i$ ，提升了杂凑输出效率。

（2）引入功能标识  $fid$  和安全参数  $c$  到初始向量  $S_0$ ，对不同功能和安全等级进行区分，确保相同消息在不同用途或不同安全等级下生成的杂凑输出不同。

在具体算法设计时，我们使用同一个 1600 比特的置换  $P$ ，通过改变安全容量  $c$  的大小，设计了 3 个安全等级分别为 512、768 和 1024 比特杂凑算法 Litchi-512、Litchi-768 和 Litchi-1024；同时设计了 Litchi-XOF 算法，安全参数  $c$  为 1024，安全强度可达到 512 比特。

综上，根据以上思路设计的 Litchi 算法上层结构，与  $\text{Sponge}$  结构相比，在达到相同的安全性时效率更高；例如在杂凑输出长度为 512 比特时，消息处理效率提升了接近一倍，而且在作为扩展输出函数时，杂凑输出可并行处理提升输出效率。

## 3.2 置换 P 设计

Litchi 杂凑算法的置换 P 基于比特切片设计，采用 SPN 结构，轮函数由常数加变换  $\iota$ 、非线性变换  $\chi$ 、线性变换  $\theta$  和换位变换  $\pi$  组成。每个部件尽可能简洁，并且在各个部件的相互配合下以较低轮数可达到高安全性。

线性变换  $\theta$  是置换 P 的最大创新点与设计核心，其基于一个带参数的 320 比特的线性变换  $\text{Theta}[t_0, t_1, t_2, t_3, t_4]$  设计， $\text{Theta}[t_0, t_1, t_2, t_3, t_4]$  实现深度与 Keccak[BDPV11, DM15] 的线性变换相同，但分支数由 4 提升到了 12。基于不同的参数，可以设计出不同的线性变换。我们围绕着线性变换  $\text{Theta}[t_0, t_1, t_2, t_3, t_4]$  进行置换 P 的设计，并精细设计了其它密码部件及相关参数。

置换 P 借鉴了 Ascon 算法[DEMS21]的设计思想，Ascon 算法的置换 P 将 320 比特的状态分为 5 个 64 比特的棱，不同棱使用不同参数控制的简洁线性变换达到扩散的目的，棱之间采用非线性变换（5 比特 S 盒）进行混淆，且 S 盒的差分 and 线性分支数都为 3，使得差分 and 线性掩码传递能达到更多的棱。

Litchi 的置换 P 将 1600 比特的状态分为 5 个 320 比特的面，不同面使用不同参数控制的简洁线性变换达到扩散的目的，面之间采用非线性变换（5 比特 S 盒）进行混淆。与 Ascon 算法置换 P 中 S 盒的设计不同，为了实现更简洁高效，我们没有采用分支数为 3 的 S 盒，而是选择了分支数为 2 的路径尽可能少的 S 盒。同时我们增加了换位变换  $\pi$ ，将不同面的 64 比特棱之间进行交换，使得不同面之间的扩散性更快，而且不论是软件实现还是硬件实现，换位变换  $\pi$  都不会增加额外代价。

通过精巧的参数选取，Litchi 的置换 P 可以在 3 轮达到全扩散，并且具有较强的抵抗差分 and 线性等攻击能力。

下面我们具体介绍轮函数各个密码部件的设计原理和密码学性质。

### 3.2.1 非线性变换设计

非线性变换  $\chi$  主要是对状态 A 的 5 个面对应位进行混淆。基于安全性

与效率的折中考量，在非线变换设计时，我们考虑了 S 盒的代数次数、最大差分概率和最大线性偏差、差分和线性的扩散性等，同时为了比特切片实现的高效性，S 盒需具有简洁的代数表达式。具体的，非线性变换  $\chi$  所用的 S 盒性质如下：

- (1) 代数次数为 3，逆 S 盒代数次数也为 3；
- (2) 最大差分概率为  $2^{-2}$ ；
- (3) 最大线性偏差为  $2^{-2}$ ；
- (4) 差分分支数为 2，差分分布表（见表 3-1）中输入输出重量均为 1 的个数为 5 个；
- (5) 线性分支数为 2，线性逼近表（见表 3-2）中输入输出重量均为 1 的个数为 5 个；
- (6) S 盒实现只需 5 个与门和 5 个异或门，具体见本文“5.2.1 软件性能测试”。

表 3-1 差分分布表：  $DDT[\alpha, \beta] = \#\{x | S(x \oplus \alpha) \oplus S(x) = \beta\}$

|    | 0  | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 | 31 |
|----|----|---|---|---|---|---|---|---|---|---|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 0  | 32 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  |
| 1  | 0  | 0 | 8 | 4 | 0 | 0 | 0 | 0 | 0 | 0 | 0  | 4  | 0  | 0  | 0  | 0  | 0  | 0  | 8  | 4  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 0  | 0  | 0  | 0  |
| 2  | 0  | 0 | 0 | 0 | 8 | 0 | 4 | 2 | 0 | 0 | 0  | 0  | 0  | 0  | 0  | 2  | 0  | 0  | 0  | 0  | 0  | 4  | 4  | 2  | 0  | 0  | 0  | 0  | 0  | 4  | 0  | 2  |
| 3  | 0  | 0 | 0 | 0 | 8 | 4 | 4 | 2 | 0 | 0 | 0  | 0  | 0  | 4  | 0  | 2  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 2  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 2  |
| 4  | 0  | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 8 | 0 | 4  | 0  | 8  | 0  | 4  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 0  | 0  | 0  | 4  | 0  |
| 5  | 0  | 0 | 0 | 0 | 0 | 4 | 0 | 0 | 0 | 4 | 4  | 0  | 0  | 0  | 4  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 0  | 0  | 0  | 4  | 4  | 0  | 0  | 0  | 4  | 0  |
| 6  | 0  | 0 | 0 | 2 | 0 | 0 | 0 | 0 | 4 | 0 | 2  | 0  | 4  | 0  | 2  | 2  | 0  | 4  | 0  | 2  | 0  | 0  | 0  | 0  | 0  | 0  | 2  | 0  | 0  | 4  | 2  | 2  |
| 7  | 0  | 4 | 0 | 2 | 0 | 0 | 0 | 0 | 4 | 0 | 2  | 0  | 4  | 4  | 2  | 2  | 0  | 0  | 0  | 2  | 0  | 0  | 0  | 0  | 0  | 0  | 2  | 0  | 0  | 0  | 2  | 2  |
| 8  | 0  | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0  | 0  | 0  | 0  | 0  | 0  | 8  | 0  | 0  | 0  | 8  | 0  | 0  | 0  | 8  | 0  | 0  | 0  | 8  | 0  | 0  | 0  |
| 9  | 0  | 0 | 0 | 2 | 0 | 0 | 0 | 2 | 0 | 0 | 4  | 2  | 0  | 0  | 4  | 2  | 0  | 0  | 4  | 2  | 0  | 0  | 4  | 2  | 0  | 0  | 0  | 2  | 0  | 0  | 0  | 2  |
| 10 | 0  | 0 | 0 | 2 | 0 | 4 | 0 | 0 | 0 | 0 | 4  | 2  | 0  | 4  | 0  | 0  | 0  | 0  | 4  | 2  | 4  | 0  | 0  | 0  | 0  | 0  | 0  | 2  | 4  | 0  | 0  | 0  |
| 11 | 0  | 0 | 0 | 0 | 0 | 0 | 0 | 2 | 0 | 0 | 0  | 0  | 0  | 0  | 4  | 2  | 4  | 4  | 0  | 0  | 0  | 0  | 4  | 2  | 4  | 4  | 0  | 0  | 0  | 0  | 0  | 2  |
| 12 | 0  | 0 | 4 | 0 | 0 | 0 | 4 | 0 | 0 | 0 | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 0  | 0  | 0  | 4  | 0  | 0  | 0  | 4  | 0  | 4  | 0  | 4  | 0  | 4  | 0  |
| 13 | 0  | 2 | 4 | 0 | 0 | 2 | 4 | 0 | 0 | 2 | 0  | 0  | 0  | 2  | 0  | 0  | 0  | 2  | 0  | 0  | 0  | 2  | 0  | 0  | 0  | 2  | 4  | 0  | 0  | 2  | 4  | 0  |
| 14 | 0  | 2 | 2 | 0 | 0 | 2 | 2 | 2 | 0 | 2 | 0  | 2  | 0  | 2  | 0  | 0  | 2  | 0  | 0  | 2  | 2  | 0  | 0  | 0  | 2  | 0  | 2  | 0  | 2  | 0  | 2  | 2  |
| 15 | 0  | 0 | 2 | 0 | 0 | 0 | 2 | 2 | 0 | 0 | 0  | 2  | 0  | 0  | 0  | 0  | 2  | 2  | 0  | 2  | 2  | 2  | 0  | 0  | 2  | 2  | 2  | 0  | 2  | 2  | 2  | 2  |
| 16 | 0  | 8 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 8 | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 8  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 8  | 0  | 0  | 0  | 0  | 0  | 0  |
| 17 | 0  | 0 | 8 | 4 | 0 | 0 | 0 | 0 | 0 | 0 | 0  | 4  | 0  | 0  | 0  | 0  | 0  | 0  | 8  | 4  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 0  | 0  | 0  | 0  |
| 18 | 0  | 0 | 0 | 0 | 8 | 4 | 4 | 2 | 0 | 0 | 0  | 0  | 0  | 4  | 0  | 2  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 2  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 2  |
| 19 | 0  | 0 | 0 | 0 | 8 | 0 | 4 | 2 | 0 | 0 | 0  | 0  | 0  | 0  | 0  | 2  | 0  | 0  | 0  | 0  | 0  | 4  | 4  | 2  | 0  | 0  | 0  | 0  | 0  | 4  | 0  | 2  |
| 20 | 0  | 0 | 0 | 0 | 0 | 4 | 0 | 4 | 0 | 4 | 0  | 4  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 0  | 4  | 0  | 4  | 0  | 4  | 0  | 0  | 0  | 0  |
| 21 | 0  | 0 | 0 | 0 | 0 | 0 | 0 | 4 | 8 | 0 | 0  | 4  | 8  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 0  | 0  | 0  | 4  | 0  | 0  | 0  | 0  |
| 22 | 0  | 4 | 0 | 2 | 0 | 0 | 0 | 0 | 4 | 0 | 2  | 0  | 4  | 4  | 2  | 2  | 0  | 0  | 0  | 2  | 0  | 0  | 0  | 0  | 0  | 0  | 2  | 0  | 0  | 0  | 2  | 2  |
| 23 | 0  | 0 | 0 | 2 | 0 | 0 | 0 | 0 | 4 | 0 | 2  | 0  | 4  | 0  | 2  | 2  | 0  | 4  | 0  | 2  | 0  | 0  | 0  | 0  | 0  | 0  | 2  | 0  | 0  | 4  | 2  | 2  |
| 24 | 0  | 4 | 0 | 0 | 0 | 4 | 0 | 0 | 0 | 4 | 0  | 0  | 0  | 4  | 0  | 0  | 0  | 4  | 0  | 0  | 0  | 4  | 0  | 0  | 0  | 4  | 0  | 0  | 0  | 4  | 0  | 0  |
| 25 | 0  | 0 | 0 | 2 | 0 | 0 | 0 | 2 | 0 | 0 | 4  | 2  | 0  | 0  | 4  | 2  | 0  | 0  | 4  | 2  | 0  | 0  | 4  | 2  | 0  | 0  | 0  | 2  | 0  | 0  | 0  | 2  |
| 26 | 0  | 0 | 0 | 2 | 0 | 0 | 0 | 0 | 0 | 0 | 4  | 2  | 0  | 0  | 0  | 0  | 0  | 4  | 2  | 4  | 4  | 0  | 0  | 0  | 0  | 0  | 2  | 4  | 4  | 0  | 0  | 0  |

|    |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|----|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 27 | 0 | 4 | 0 | 0 | 0 | 0 | 0 | 2 | 0 | 4 | 0 | 0 | 0 | 0 | 4 | 2 | 4 | 0 | 0 | 0 | 0 | 0 | 4 | 2 | 4 | 0 | 0 | 0 | 0 | 0 | 0 | 2 |
| 28 | 0 | 2 | 0 | 2 | 0 | 2 | 0 | 2 | 0 | 2 | 0 | 2 | 0 | 2 | 0 | 2 | 0 | 2 | 0 | 2 | 0 | 2 | 0 | 2 | 0 | 2 | 0 | 2 | 0 | 2 | 0 | 2 |
| 29 | 0 | 0 | 0 | 2 | 0 | 0 | 0 | 2 | 0 | 0 | 0 | 2 | 0 | 0 | 0 | 2 | 4 | 0 | 0 | 2 | 4 | 0 | 0 | 2 | 4 | 0 | 0 | 2 | 4 | 0 | 0 | 2 |
| 30 | 0 | 0 | 2 | 2 | 0 | 0 | 2 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 2 | 2 | 2 | 0 | 0 | 2 | 2 | 0 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 0 |
| 31 | 0 | 2 | 2 | 2 | 0 | 2 | 2 | 0 | 0 | 2 | 0 | 0 | 0 | 2 | 0 | 2 | 2 | 0 | 0 | 2 | 0 | 0 | 2 | 2 | 0 | 2 | 2 | 2 | 0 | 2 | 0 | 2 |

表 3-2 线性逼近表:  $LAT[\alpha, \beta] = \#\{x | \alpha^T \cdot x = \beta^T \cdot S(x)\} - 16$

|    | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 | 31 |    |
|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 0  | 16 | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  |    |
| 1  | 0  | 0  | 8  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 4  | 0  | 0  | -4 | 4  | 0  | 0  | 0  | 0  | 4  | -4 | 4  | 4  | 4  | -4 | 0  | 0  | 0  | 0  | -4 | -4 |    |
| 2  | 0  | 0  | 0  | 0  | 8  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 8  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 4  | 4  | -4 | 0  | 0  | 0  | 0  | -4 | -4 | 4  | -4 |    |
| 3  | 0  | 0  | 8  | 0  | 0  | 0  | 8  | 0  | 0  | 0  | -4 | -4 | 0  | 0  | 4  | 4  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | -4 | 4  | 0  | 0  | 4  | -4 | 0  | 0  |    |
| 4  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 8  | 8  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 8  | -8 | 0  | 0  | 0  |    |
| 5  | 0  | 0  | 8  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 4  | 0  | 0  | 4  | -4 | 0  | 0  | 0  | 0  | -4 | 4  | -4 | -4 | 4  | -4 | 0  | 0  | 0  | 0  | 4  | 4  |    |
| 6  | 0  | 0  | 0  | 0  | 8  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 8  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | -4 | -4 | -4 | 4  | 0  | 0  | 0  | 0  | 4  | 4  | 4  | -4 |    |
| 7  | 0  | 0  | -8 | 0  | 0  | 0  | 8  | 0  | 0  | 0  | 4  | 4  | 0  | 0  | 4  | 4  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | -4 | 0  | 0  | 4  | -4 | 0  | 0  |    |
| 8  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 8  | 8  | 8  | -8 | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  |    |
| 9  | 0  | 0  | 0  | 0  | 0  | 0  | 8  | 0  | 0  | 0  | 4  | -4 | 0  | 0  | -4 | -4 | 4  | -4 | 4  | 4  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 4  | -4 | 4  | 0  | 0  |
| 10 | 0  | 0  | 0  | 0  | 8  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | -8 | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 4  | 4  | -4 | 0  | 0  | 0  | 0  | 4  | 4  | -4 | 4  |    |
| 11 | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | -4 | 4  | 0  | 0  | 4  | -4 | 4  | -4 | 4  | 4  | 4  | -4 | 4  | 4  | 0  | 0  | -4 | -4 | 0  | 0  | 4  | 4  |    |
| 12 | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 8  | -8 | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 8  | 8  | 0  | 0  | 0  | 0  | 0  | 0  |    |
| 13 | 0  | 0  | 0  | 0  | 0  | 0  | -8 | 0  | 0  | 0  | 4  | -4 | 0  | 0  | 4  | 4  | 4  | -4 | 4  | 4  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 4  | 4  | -4 | 0  | 0  |    |
| 14 | 0  | 0  | 0  | 0  | -8 | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 8  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 4  | 4  | -4 | 0  | 0  | 0  | 0  | 4  | 4  | 4  | -4 |    |
| 15 | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | -4 | 0  | 0  | 4  | -4 | -4 | 4  | -4 | -4 | 4  | -4 | 4  | 4  | 0  | 0  | 4  | 4  | 0  | 0  | 4  | 4  |    |
| 16 | 0  | 8  | 0  | 0  | 0  | 4  | 0  | -4 | 0  | 0  | 4  | 4  | 4  | 0  | 0  | -4 | 0  | 0  | 0  | 0  | 0  | 4  | 0  | 4  | -4 | 4  | 0  | 0  | 0  | -4 | -4 | 0  |    |
| 17 | 0  | 8  | 0  | 8  | 0  | -4 | 0  | 4  | 0  | 0  | 0  | 0  | -4 | 0  | 4  | 0  | 0  | 0  | 0  | 0  | -4 | 0  | 4  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 0  | -4 |    |
| 18 | 0  | 8  | 0  | 0  | 0  | 4  | 0  | 4  | 0  | 0  | -4 | -4 | 4  | 0  | 0  | 4  | 0  | 0  | 0  | 0  | 4  | 0  | -4 | 0  | 4  | -4 | 0  | 0  | -4 | 0  | 0  | 4  |    |
| 19 | 0  | -8 | 0  | 8  | 0  | 4  | 0  | 4  | 0  | 0  | 0  | 0  | 4  | 0  | 4  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 0  | 4  | 0  | 0  | 0  | 0  | -4 | 0  | -4 | 0  |    |
| 20 | 0  | 0  | 0  | 8  | 0  | 4  | 0  | -4 | 4  | 4  | 0  | 0  | 0  | 4  | -4 | 0  | 0  | 0  | 0  | 0  | 0  | -4 | 0  | -4 | 0  | 0  | -4 | 4  | 4  | 0  | 0  | 4  |    |
| 21 | 0  | 0  | 0  | 0  | 0  | -4 | 0  | 4  | 4  | 4  | 4  | 4  | 0  | -4 | 0  | 4  | 0  | 0  | 0  | 0  | 4  | 0  | -4 | 0  | -4 | 4  | -4 | 4  | -4 | 0  | 4  | 0  |    |
| 22 | 0  | 0  | 0  | -8 | 0  | 4  | 0  | 4  | 4  | 4  | 4  | 0  | 0  | 0  | 4  | 4  | 0  | 0  | 0  | 0  | -4 | 0  | 4  | 0  | 0  | 0  | -4 | 4  | 0  | 4  | -4 | 0  |    |
| 23 | 0  | 0  | 0  | 0  | 0  | 4  | 0  | 4  | -4 | -4 | 4  | 4  | 0  | 4  | 0  | 4  | 0  | 0  | 0  | 0  | 0  | -4 | 0  | -4 | -4 | 4  | 4  | -4 | 0  | 4  | 0  | 4  |    |
| 24 | 0  | 0  | 0  | 0  | 0  | 4  | 0  | 4  | 0  | 0  | -4 | 4  | -4 | 0  | 0  | -4 | 4  | 4  | -4 | 4  | 4  | 0  | -4 | 0  | 0  | 0  | 4  | 4  | 4  | 4  | 0  | 0  | -4 |
| 25 | 0  | 0  | 0  | 0  | 0  | -4 | 0  | 4  | 0  | 0  | 0  | 0  | 4  | 0  | -4 | 0  | 0  | 8  | 0  | 8  | -4 | 0  | 4  | 0  | 0  | 0  | 0  | 0  | 0  | -4 | 0  | 4  | 0  |
| 26 | 0  | 0  | 0  | 0  | 0  | 4  | 0  | -4 | 0  | 0  | 4  | -4 | -4 | 0  | 0  | 4  | 4  | 4  | -4 | 4  | 0  | 4  | 0  | 4  | 0  | 0  | -4 | -4 | 0  | 4  | 4  | 0  | 0  |
| 27 | 0  | 0  | 0  | 0  | 0  | 4  | 0  | 4  | 0  | 0  | 0  | 0  | -4 | 0  | -4 | 0  | -8 | 0  | 8  | 0  | 0  | 4  | 0  | 4  | 0  | 0  | 0  | 0  | 4  | 0  | 4  | 0  | 0  |
| 28 | 0  | 0  | 0  | 0  | 0  | -4 | 0  | -4 | -4 | 4  | 0  | 0  | 0  | 4  | 4  | 0  | -4 | 4  | 4  | 4  | 4  | 0  | -4 | 0  | 4  | 4  | 0  | 0  | 0  | 4  | -4 | 0  | 0  |
| 29 | 0  | 0  | 0  | 0  | 0  | 4  | 0  | -4 | -4 | 4  | -4 | 4  | 0  | -4 | 0  | 4  | 0  | 0  | 0  | 0  | -4 | 0  | 4  | 0  | 4  | 4  | 4  | 4  | -4 | 0  | 4  | 0  | 0  |
| 30 | 0  | 0  | 0  | 0  | 0  | -4 | 0  | 4  | -4 | 4  | 0  | 0  | 0  | 4  | -4 | 0  | 4  | -4 | -4 | -4 | 0  | 4  | 0  | 4  | 4  | 4  | 0  | 0  | 4  | 0  | 0  | 4  | 0  |
| 31 | 0  | 0  | 0  | 0  | 0  | -4 | 0  | -4 | 4  | -4 | -4 | 4  | 0  | 4  | 0  | 4  | 0  | 0  | 0  | 0  | 0  | 4  | 0  | 4  | -4 | -4 | 4  | 4  | 0  | 4  | 0  | 4  | 0  |

为了增强抵抗代数攻击的相关能力，设计时要求 S 盒的代数次数不低于 3。由于置换 P 的线性变换 $\theta$ 是对状态 A 的每个面进行操作，而非线性变换 $\chi$ 主要是对状态 A 的 5 个面对应位进行混淆，所以如果 S 盒具有较好的扩散性，可增强置换 P 抵抗差分和线性攻击的能力；但若 S 盒差分分支数和线性分支数都达到 3，则会大幅增加 S 盒表达式的复杂度，需要付出较高的实现代价，出于折中考虑，我们设计的 S 盒虽然分支数不能达到 3，但是分支数为 2 的点尽可能少。

### 3.2.2 线性变换设计

线性变换 $\theta$ 是将状态 A 的每个面  $A[i]$  分别经过线性变换 $\theta_i$  进行更新,  $0 \leq i \leq 4$ 。线性变换 $\theta_i$  是一个 320 比特的线性置换, 由可变参数的线性置换  $\text{Theta}[t_0, t_1, t_2, t_3, t_4]$  确定, 其中  $t_i$  为 0 到 63 的整数。

$V = \text{Theta}[t_0, t_1, t_2, t_3, t_4](U)$  为参数  $(t_0, t_1, t_2, t_3, t_4)$  控制下的 320 比特线性置换, 将 320 比特输入和输出分为 5 个 64 比特棱, 即  $U = U_0 || U_1 || U_2 || U_3 || U_4$ ,  $V = V_0 || V_1 || V_2 || V_3 || V_4$ , 使用环  $\text{GF}(2)[x]/(x^{64}+1)$  上的多项式可以表示为:

$$V_i = U_i + \sum_{j=0}^4 (x^{-t_i} + x^{t_j}) U_j, 0 \leq i \leq 4$$

Theta 满足如下性质:

(1) 满足对称性, 即  $\text{Theta}^T = \text{Theta}$ ;

(2) 满足可逆性; 其逆变换为

$$U_i = V_i + (\sum_{k=0}^4 x^{-t_k} + \sum_{k=0}^4 x^{t_k} + \sum_{k=0}^4 x^{-t_k} \sum_{k=0}^4 x^{t_k})^{-1} \sum_{j=0}^4 (1 + (1 + \sum_{k=0}^4 x^{t_k}) x^{-t_i} + (1 + \sum_{k=0}^4 x^{-t_k}) x^{t_j} + x^{-t_i+t_j}) V_j。$$

(3) 在选择特定的参数时, 差分分支数和线性分支数都可达 12。 $\theta_i$  均可达到差分分支数和线性分支数为 12。

为了分支数达到 12,  $\text{Theta}[t_0, t_1, t_2, t_3, t_4]$  中参数的选取需满足如下要求:

- ①  $t_i \neq 2t_j \bmod 64$ , 对任意互异的  $i, j$ , 且  $0 \leq i, j \leq 4$ ;
- ②  $t_i \neq -t_j \bmod 64$ , 对任意的  $i$  和  $j$ , 且  $0 \leq i, j \leq 4$ ;
- ③  $t_i + t_j \neq 2t_k \bmod 64$ , 对任意互异的  $i, j, k$ , 且  $0 \leq i, j, k \leq 4$ ;
- ④  $t_i + t_j \neq t_k + t_l \bmod 64$ , 对任意互异的  $i, j, k, l$ , 且  $0 \leq i, j, k, l \leq 4$ 。

Theta 的硬件实现深度与 Keccak 线性变换 $\theta$ 相同, 软件实现只需 18 个 64 位的异或和 10 个循环移位, 且分支数由 4 提升到了 12, 大幅提升置换 P 抵抗差分、线性等攻击的能力。文献[HWL24]提出的 STCPM 方案, 虽然差分和线性分支数也可以达到 12, 但是计算深度和复杂度比 Theta 变换都要高。

### 3.2.3 换位变换设计

换位变换  $\pi$  主要作用是将棱进行换位操作, 使得状态 A 同一个面上的

5 个棱经换位变换  $\pi$  后处于不同的 5 个面，达到快速扩散的目的。同时，为了不破坏线性变换  $\theta_i$  的分支数，状态 A 同一个面上的 5 个棱经过换位变换  $\pi$  后任意 2 个棱不能处于同一列，即不能有任何 2 个棱中对应位经过同一个 S 盒变换。

换位变换  $\pi$  中棱位置的变换可看作如下矩阵运算，矩阵运算为域 GF(5) 的运算，棱的坐标  $(i, j)$  经过  $\pi$  变换后位置变为  $(i', j')$ 。计算方式为：

$$\begin{pmatrix} i' \\ j' \end{pmatrix} = M \begin{pmatrix} i \\ j \end{pmatrix} = \begin{pmatrix} 4 & 4 \\ 1 & 3 \end{pmatrix} \begin{pmatrix} i \\ j \end{pmatrix}$$

换位变换  $\pi$  满足如下性质：

- (1) M 可逆，逆矩阵  $M^{-1} = \begin{pmatrix} 1 & 2 \\ 3 & 3 \end{pmatrix}$ 。
- (2) 换位变换  $\pi$  转置变换对应的棱位置变换矩阵为  $\begin{pmatrix} 1 & 2 \\ 3 & 3 \end{pmatrix}$ 。
- (3) 矩阵 M 的阶为 24，即  $\begin{pmatrix} 4 & 4 \\ 1 & 3 \end{pmatrix}^{24} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$ ，其中  $2 \times 2$  可逆矩阵的最大阶为 24。
- (4) 同一行的任意两个点经过  $\pi$  变换后不能到同一行，且不能在上一列。即横坐标相同，纵坐标不同的任意两个点，经过  $\pi$  变换后横坐标和纵坐标均不相同。容易验证此性质：假设横坐标相同，纵坐标不同的任意两个点为  $(u, v)$  和  $(u, v')$ ，经过  $\pi$  变换后坐标为  $(4u+4v, u+3v)$  和  $(4u+4v', u+3v')$ ，可推得  $4u+4v \neq 4u+4v'$ ， $u+3v \neq u+3v'$ 。

- (5) 同一列的任意两个点经过  $\pi$  变换后不能到同一行，且不能在上一列。即横坐标不同，纵坐标相同的任意两个点，经过  $\pi$  变换后横坐标和纵坐标均不相同。容易验证此性质：假设横坐标不同，纵坐标相同的任意两个点为  $(u, v)$  和  $(u', v)$ ，经过  $\pi$  变换后坐标为  $(4u+4v, u+3v)$  和  $(4u'+4v, u'+3v)$ ，可推得  $4u+4v \neq 4u'+4v$ ， $u+3v \neq u'+3v$ 。

### 3.2.4 常数加设计

每轮变换中引入轮常数主要是为了抵抗滑动、旋转攻击等，常数选取自然常数 e 的小数部分，显示出常数选取没有隐藏任何后门。轮常数更新方式为有限域  $\text{GF}(2)[x]/(x^{64}+x^{63}+x^{62}+x^{54}+x^{40}+x^{35}+x^{15}+x+1)$  上的运算，与线性变换  $\theta$  在环  $\text{GF}(2)[x]/(x^{64}+1)$  上的运算不同，避免两者结合的相关攻击。

轮常数每轮只取 64 比特，兼顾了实现的效率和资源占用。

### 3.3 参数选择依据

我们根据前面的提到的各个密码部件需要满足的性质，选择了相应的参数，并结合安全性分析，进一步从满足性质的参数中筛选出抗差分、线性等能力更强的参数。

## 4 安全性声明与分析

### 4.1 理论安全性

#### 4.1.1 上层结构安全性分析

假设 Litchi 算法上层结构中容量为  $c$ ，杂凑输出长度为  $|h|$  比特， $l$  为第二原像查询消息的块数， $t$  为计数器比特长度，则 Litchi 抵抗各种攻击的复杂度如下：

- (1) 碰撞攻击： $\min(O(2^{|h|/2}), O(2^{c/2}))$ ;
- (2) 原像攻击： $\min(O(2^{|h|}), O(2^c))$ ;
- (3) 第二原像攻击： $\min(O(2^{|h|}), O(2^{c-\log_2 l}))$ ;
- (4) 量子碰撞攻击： $\min(O(2^{|h|/4}), O(2^{c/4}))$ ;
- (5) 量子原像攻击： $\min(O(2^{|h|/2}), O(2^{c/2}))$ ;
- (6) 量子第二原像攻击： $\min(O(2^{|h|/2}), O(2^{(c-\log_2 l)/2}))$ ;

(7) 若置换  $P$  是一个随机置换，即使敌手可对该置换进行任意询问，则将 Litchi 算法与随机预言机区分的复杂度为  $\min(O(2^{(c-t)/2}), 2^{1600-\min(c, |h|)})$ 。其中，Litchi 上层结构作为杂凑使用时，输出仅有一块  $h_1$ ，计数器值可看作常数，所以  $t = 0$ ；作为 XOF 使用时，输出长度小于  $2^{64}$  比特， $t \leq 64$ 。

Litchi 上层结构与 Sponge- $F^{P, pd}$  在作为杂凑（即固定输出长度输出）使用时，二者完全相同，所以上述结论的分析证明过程具体参见文献 [GHJ+25]。

Litchi 上层结构与 Sponge- $F^{P, pd}$  结构在作为 XOF 使用时存在较大的差异：Litchi 上层结构采用基于计数器的并行方式，而 Sponge- $F^{P, pd}$  结构采用串行方式。因此，这里我们只分析 Litchi 作为 XOF 函数使用时的安全性。

Litchi 上层结构作为 XOF 使用时,  $h = \text{MSB}_{|h|}(h_1 || h_2 || \dots || h_{\lceil |h|/c \rceil})$ , 此时要找到  $h$  的攻击, 必须找到对  $h_1$  的攻击, 例如要找到  $h$  的原像, 必须找到  $h_1$  的原像。因为使用的是计数器模式,  $P$  的输入都不相同, 且  $P$  为随机置换, 即每个  $h_i$  都是随机独立的, 获得其它  $h_i$  的值无法得到任何  $h_1$  的相关信息。

在 Litchi 处理最后一个消息块时, 令  $X = S_{l-1} \oplus (M_{l-1} || 0^c)$ , 第  $i$  个输出块  $h_i$  可视为关于  $X$  的函数, 记为  $h_i = f_i(X)$ , 其中  $f_i(X) = \text{LSB}_c(P(X \oplus (0^r || [i]_c)) \oplus (X \oplus (0^r || [i]_c)))$ ,  $i = 1, 2, \dots, s = \lceil |h|/c \rceil$ 。因此, 对于固定的  $X$ , 在理想置换模型下这些  $h_i$  都是独立的随机值, 并且由  $X$  可直接计算得到全部的输出值  $h_i$ 。

此外, 根据 Litchi 算法的设计,  $X$  的具体值完全由消息  $M$  来决定, 记为  $X = g(M)$ 。因此,  $h_i$  还可以写为  $h_i = f_i(g(M))$ ,  $i = 1, 2, \dots, s = \lceil |h|/c \rceil$ 。

基于上述分析, 若 Litchi 仅作为固定输出长度的杂凑算法 (仅有 1 个输出块), 则采用上述记号的输出块为  $h_1 = f_1(g(M))$ ; 若 Litchi 作为 XOF 函数输出  $s$  个块, 则 Litchi-XOF 相当于独立执行了  $s$  个固定输出长度的 Litchi 算法 (即 Litchi-XOF 由  $s$  个独立算法  $f_i(g(M))$  组成), 其中每个算法采用不同的  $i$  来实现分割。在理想置换模型下, 这种设计可实现完美的独立性分割。上述性质使得 Litchi 作为 XOF 使用时的攻击难度将超过固定输出长度的杂凑算法。下面我们将分别从抗原像、抗碰撞、抗第二原像攻击、以及不可分离性等安全性方面具体分析 Litchi-XOF 与 Litchi 杂凑算法之间的关系。

### (1) 抗原像攻击

从直观上, 如果给定 Litchi-XOF 的  $s$  个独立输出块  $h_1 h_2 \dots h_s$ , 能够找到  $M$  使得  $f_i(g(M)) = h_i (i = 1, 2, \dots, s)$  同时成立, 则给定 1 个输出块  $h_1$ , 可以更容易地找到  $M$ , 使得只有 1 个  $f_1(g(M)) = h_1$  成立。反之则不一定。因此, Litchi-XOF 的抗原像攻击能力应该比 Litchi 杂凑算法更强。下面给出严格的分析。

给定 Litchi-XOF 的输出块  $h_1 h_2 \dots h_s$ , 若存在  $q$  次查询的原像攻击者  $\mathcal{A}$  可输出一个原像  $M$ , 使得  $f_i(g(M)) = h_i$  对任意  $i = 1, 2, \dots, s$  都成立。则可构

造攻击者 $\mathcal{B}$ ：先调用 $\mathcal{A}$ 得到原像 $M$ （记原像有 $l$ 个消息块），再计算 $g(M)$ ，从而得到一个 $X$ 使得 $f_i(X) = h_i$ ，即 $\mathcal{B}$ 可以解决如下困难问题**HP**：寻找 $X$ ，使得 $f_i(X) = h_i, \forall i = 1, 2, \dots, s$ 。这里 $\mathcal{B}$ 的运行时间与 $\mathcal{A}$ 基本相当，而查询次数为 $q + l$ 。因此， $q$ 次查询者 $\mathcal{A}$ 的原像攻击优势 $Adv_{XOF}^{pre}(\mathcal{A}, q)$ 不超过 $q + l$ 次查询者 $\mathcal{B}$ 在解决该困难问题**HP**时的优势 $Adv^{HP}(\mathcal{B}, q + l)$ 。

在理想置换假设下， $\mathcal{B}$ 经过1次查询找到 $X$ 满足某个关系式 $f_i(X) = h_i$ 的概率不超过 $\frac{2^{b-c}}{2^{b-q-l}}$ ，而这 $s$ 个关系式是完全独立的，则找到的 $X$ 同时满足这 $s$ 个关系式的概率不超过 $\left(\frac{2^{b-c}}{2^{b-q-l}}\right)^s$ 。因此，对全部 $q + l$ 次查询取联合界得到 $\mathcal{B}$ 在解决该困难问题**HP**时的优势 $Adv^{HP}(\mathcal{B}, q + l) \leq (q + l) \cdot \left(\frac{2^{b-c}}{2^{b-q-l}}\right)^s$ 。

进一步得到

$$Adv_{XOF}^{pre}(\mathcal{A}, q) \leq Adv^{HP}(\mathcal{B}, q + l) \leq (q + l) / \left(2^c - \frac{q + l}{2^{b-c}}\right)^s$$

对于 Litchi 杂凑算法，抗原像攻击的优势上界为 $Adv_{HASH}^{pre}(\mathcal{A}, q) \leq (q + l) / \left(2^c - \frac{q+l}{2^{b-c}}\right)$ ，正好对应于上式中 $s = 1$ 的情况。因此，Litchi-XOF 的抗原像攻击能力不低于 Litchi 杂凑算法。

## （2）抗碰撞攻击

假设 Litchi-XOF 抗第二原像攻击能力低于 Litchi 杂凑算法，则敌手可通过寻找消息  $M$  的 Litchi-XOF 第二原像，得到  $M$  的 Litchi 杂凑算法第二原像。具体方法如下：

通过消息  $M$ ，计算得到  $h_1 || h_2 || \dots || h_s = \text{Litchi-XOF}(M)$ ，寻找  $M'$  使得  $M' \neq M$  且  $\text{Litchi-XOF}(M) = \text{Litchi-XOF}(M')$ ，则  $M'$  也是 Litchi 作为杂凑算法时  $M$  的第二原像。

这样就以计算 Litchi-XOF 第二原像的代价得到 Litchi 杂凑算法的第二原像，这与 Litchi-XOF 的抗第二原像攻击能力低于 Litchi 杂凑算法相矛盾。因此，Litchi-XOF 抗第二原像攻击能力不低于 Litchi 杂凑算法。

## （3）抗第二原像攻击

若找到消息碰撞对  $M_1$  和  $M_2$ ，满足  $\text{Litchi-XOF}(M_1) = \text{Litchi-XOF}(M_2)$ ，

则  $M_1$  和  $M_2$  必然也是 Litchi 作为杂凑时的碰撞对，所以 Litchi-XOF 抗碰撞攻击能力不低于 Litchi 杂凑算法。

#### (4) 不可分离性

将 Litchi-XOF 结构中计数器看作  $t$  比特的消息，则此时安全容量变为  $c-t$ ，根据文献[GHJ+25]不可分离的结论可得，Litchi-XOF 结构的不可分离安全界至少为  $\min(O(2^{(c-t)/2}), 2^{1600-\min(c, |h|)})$ 。

### 4.1.2 置换 P 安全性分析

#### 4.1.2.1. 差分分析

差分分析是对置换 P 最重要的分析，同时也是我们对线性变换  $\theta_i$  参数选择和换位变换  $\pi$  设计的重要依据。

由于差分分析时不需要考虑常数加变换  $\iota$ ，所以置换 P 可只考虑由  $\pi \circ \theta \circ \chi$  组成的轮函数  $f$ ，对于第  $i$  轮的  $\chi$  变换的输入差分设为  $\Delta X_i$ ， $\theta$  变换的输入差分设为  $\Delta Y_i$ （即  $\chi$  变换的输出差分）， $\pi$  变换的输入差分设为  $\Delta Z_i$ （即  $\theta$  变换的输出差分）。

$$\text{即 } \Delta X_i \xrightarrow{\chi} \Delta Y_i \xrightarrow{\theta} \Delta Z_i \xrightarrow{\pi} \Delta X_{i+1}。$$

#### (1) 2 轮差分分析

因为所有  $\theta_i$  变换的差分分支数都为 12，并且经过  $\pi$  变换同一个面上的任意两个棱不在同一列，所以很容易证明置换 P 的 2 轮最小差分活动 S 盒为 12。通过计算机搜索，同样可得置换 P 的 2 轮最小差分活动 S 盒为 12。

#### (2) 3 轮差分分析

对于 3 轮的差分分析，我们使用 MILP 方式进行差分活动 S 盒搜索，S 盒使用差分分布表刻画，线性变换使用比特异或方式刻画。直接搜索 3 轮效率很低，我们使用一些技巧来搜索得到 3 轮最小差分活动 S 盒的下界和上界。

我们主要通过对 2 轮的详细差分活动 S 盒搜索得到 3 轮最小差分活动 S 盒的下界。具体的，通过搜索输入为  $\Delta Y_i$ ，输出为  $\Delta X_{i+1}$  的 2 轮最小差分活动 S 盒。

$$\Delta Y_i \xrightarrow{\theta} \Delta Z_i \xrightarrow{\pi} \Delta X_{i+1}$$

为了方便表示,我们将 $\Delta Y_{ij}$ 表示 $\Delta Y_i$ 的第 $j$ 个面。我们按 $\Delta Y_{ij}$ 是否为 0, 分为 32 种情况。然后对这 32 种情况分别求 2 轮的活动 S 盒个数, 具体见表 4-1, 其中 0 表示 $\Delta Y_{ij}$ 为 0, 1 表示 $\Delta Y_{ij}$ 非 0。由于复杂度过高, 当 $\Delta Y_i$ 中非 0 输入差分面数大于 3 时, 无法通过搜索得到 2 轮最小差分活动 S 盒的个数, 利用定理 4-1, 我们给出保守的活动 S 盒下界值。

表 4-1 2 轮差分活动 S 盒的详细值

| $\Delta Y_{i,0}, \Delta Y_{i,1}, \Delta Y_{i,2},$<br>$\Delta Y_{i,3}, \Delta Y_{i,4}$ | 最小差分活动<br>S 盒数 | $\Delta Y_{i,0}, \Delta Y_{i,1}, \Delta Y_{i,2},$<br>$\Delta Y_{i,3}, \Delta Y_{i,4}$ | 最小差分活动<br>S 盒数 |
|---------------------------------------------------------------------------------------|----------------|---------------------------------------------------------------------------------------|----------------|
| 0,0,0,0,0                                                                             | 0              | 0,0,1,1,1                                                                             | 23             |
| 0,0,0,0,1                                                                             | 12             | 0,1,0,1,1                                                                             | 24             |
| 0,0,0,1,0                                                                             | 12             | 0,1,1,0,1                                                                             | 23             |
| 0,0,1,0,0                                                                             | 12             | 0,1,1,1,0                                                                             | 23             |
| 0,1,0,0,0                                                                             | 12             | 1,0,0,1,1                                                                             | 23             |
| 1,0,0,0,0                                                                             | 12             | 1,0,1,0,1                                                                             | 23             |
| 0,0,0,1,1                                                                             | 18             | 1,0,1,1,0                                                                             | 23             |
| 0,0,1,0,1                                                                             | 18             | 1,1,0,0,1                                                                             | 24             |
| 1,0,0,1,0                                                                             | 18             | 1,1,0,1,0                                                                             | 24             |
| 0,1,0,0,1                                                                             | 18             | 1,1,1,0,0                                                                             | 24             |
| 0,1,0,1,0                                                                             | 18             | 0,1,1,1,1                                                                             | $\geq 24$      |
| 0,1,1,0,0                                                                             | 18             | 1,0,1,1,1                                                                             | $\geq 24$      |
| 1,0,0,0,1                                                                             | 18             | 1,1,0,1,1                                                                             | $\geq 24$      |
| 0,0,1,1,0                                                                             | 18             | 1,1,1,0,1                                                                             | $\geq 24$      |
| 1,0,1,0,0                                                                             | 18             | 1,1,1,1,0                                                                             | $\geq 24$      |
| 1,1,0,0,0                                                                             | 18             | 1,1,1,1,1                                                                             | $\geq 24$      |

从表 4-1 可以看出, 当 $\Delta Y_i$ 中非 0 输入差分面数大于 2 时,  $i$  轮和  $i+1$  轮连续 2 轮差分活动 S 盒的最小值为 23。

**定理 4-1:** 假设 $\Delta Y_i$ 中原来有  $t$  个面输入差分非 0, 则再增加一个输入

差分非 0 的面后，2 轮的最小差分活动 S 盒数不会变小。

证明：假设 $\Delta Y_i$  新增一个面的输入差分后为 $\Delta Y'_i$ ，令 $\Delta X_{i+1}=\pi(\theta(\Delta Y_i))$ ， $\Delta X'_{i+1}=\pi(\theta(\Delta Y'_i))$ 。由于 $\Delta Y'_i$  比 $\Delta Y_i$  新增一个面的输入差分，所以根据  $\pi$  和  $\theta$  的性质， $\Delta X'_{i+1}$  是在 $\Delta X_{i+1}$  的基础上增加了非 0 的棱，所以 $\Delta Y'_i$  的活动 S 盒数不小于 $\Delta Y_i$  的活动 S 盒数， $\Delta X'_{i+1}$  的活动 S 盒数不小于 $\Delta X_{i+1}$  的活动 S 盒数，即可得 2 轮的最小差分活动 S 盒数不会变小。■

下面我们证明当第  $i$  轮输入差分的面数小于 3 个时， $i$  轮、 $i+1$  轮和  $i+2$  轮连续 3 轮最小差分活动 S 盒数至少为 24。由于直接搜索时间复杂度过高，我们进行如下 2 轮搜索：

$$\Delta Y_i \xrightarrow{\theta} \Delta Z_i \xrightarrow{\pi} \Delta X_{i+1} \xrightarrow{\chi} \Delta Y_{i+1}$$

搜索输入为 $\Delta Y_i$  输出为 $\Delta Y_{i+1}$  的差分对，为了得到精细的搜索结果，遍历搜索 $\Delta Y_i$  和 $\Delta Y_{i+1}$  面重量（我们称 $\Delta Y_i$  中所有差分非 0 的面的个数为 $\Delta Y_i$  的面重量）为 1 和 2 的所有可能值，共  $15 \times 15 = 225$  种情况，可以求得这 225 种情况的最小活动 S 盒都不小于 23，事实上，由于时间关系，我们在下界值达到 23 时即停止搜索，实际下界值可能不低于 23；我们搜索得到的真实差分路径存在时最小差分活动 S 盒数为 26，此时 $\Delta Y_i$  的面重量为 1， $\Delta Y_{i+1}$  的面重量为 2。

由上述分析可知，当 $\Delta Y_i$  面重量不大于 2， $\Delta Y_{i+1}$  面重量不大于 2，连续 3 轮差分活动 S 盒不小于 24。

综上，通过分析以下 3 种情况，可得置换 P 连续 3 轮最小差分活动 S 盒数不少于 24。

1) 当 $\Delta Y_i$  面重量大于 2 时，由表 4-1 可知， $i$  轮和  $i+1$  轮线性活动 S 盒不小于 23，可得连续 3 轮差分活动 S 盒不小于 24；

2) 当 $\Delta Y_i$  面重量不大于 2， $\Delta Y_{i+1}$  面重量大于 2，由表 4-1 可知， $i+1$  轮和  $i+2$  轮线性活动 S 盒不小于 23，可得连续 3 轮差分活动 S 盒不小于 24；

3) 当 $\Delta Y_i$  面重量不大于 2， $\Delta Y_{i+1}$  面重量不大于 2，则连续 3 轮线性活动 S 盒不小于 24。

事实上，对于 3 轮的差分活动 S 盒，上述 24 个活动 S 盒的计算非常保守，我们对于  $i$ 、 $i+1$  和  $i+2$  连续 3 轮的活动 S 盒进行 MILP 搜索，经过多种情况的尝试以及长时间的搜索，我们发现当  $\Delta Y_i$  只有 1 个面输入差分非 0，且此面的输入差分和输出差分均为 6 时，得到 3 轮真实路径的最小活动 S 盒数为 64 个，即 3 轮差分活动 S 盒的上界为 64，具体见表 4-2。我们猜测 64 可能是 3 轮最小差分活动 S 盒数的下界。

表 4-2 3 轮差分活动 S 盒的详细值

| $\Delta Y_{i,0}, \Delta Y_{i,1}, \Delta Y_{i,2}, \Delta Y_{i,3}, \Delta Y_{i,4}$ | 最小差分活动 S 盒数 | 每轮差分活动 S 盒个数 |
|----------------------------------------------------------------------------------|-------------|--------------|
| 1,0,0,0,0                                                                        | 65          | 6---6---53   |
| 0,1,0,0,0                                                                        | 66          | 6---6---54   |
| 0,0,1,0,0                                                                        | 66          | 6---6---54   |
| 0,0,0,1,0                                                                        | 64          | 6---6---52   |
| 0,0,0,0,1                                                                        | 69          | 6---6---57   |

综上所述，我们得到的置换 P 差分活动 S 盒数如表 4-3 所示。

表 4-3 置换 P 差分活动 S 盒数

| 轮数          | 1 | 2  | 3       |
|-------------|---|----|---------|
| 最小差分活动 S 盒数 | 1 | 12 | [24,64] |

#### 4.1.2.2. 线性分析

由于线性分析时不需要考虑常数加变换  $\iota$ ，所以置换 P 可只考虑由  $\pi \circ \theta \circ \chi$  组成的轮函数  $f$ ，则对于线性掩码的计算可表示为  $\Gamma X_{i+1} = \theta^T (\pi^T (\chi^{-1} (\Gamma X_i)))$ 。由 3.2.2 和 3.2.3 节可知， $\theta^T = \theta$ ， $\pi^T = \pi^{-1}$ ，所以可得  $\Gamma X_{i+1} = \theta (\pi^{-1} (\chi^{-1} (\Gamma X_i)))$ 。对于第  $i$  轮的  $\chi^{-1}$  变换的输入掩码设为  $\Gamma X_i$ ， $\pi^{-1}$  变换的输入掩码设为  $\Gamma Z_i$ （即  $\chi^{-1}$  变换的输出掩码）， $\theta$  变换的输入掩码设为  $\Gamma Y_i$ （即  $\pi^{-1}$  变换的输出掩码）。

$$\text{即 } \Gamma X_i \xrightarrow{\chi^{-1}} \Gamma Z_i \xrightarrow{\pi^{-1}} \Gamma Y_i \xrightarrow{\theta} \Gamma X_{i+1}。$$

##### (1) 2 轮线性分析

因为所有  $\theta_i$  变换的线性分支数都为 12，并且经过  $\pi^{-1}$  变换同一个面上的任意两个棱不在同一列，很容易证明置换 P 的 2 轮线性最小活动 S 盒为

12. 通过计算机搜索，同样可得到置换 P 的 2 轮最小线性活动 S 盒为 12。

### (2) 3 轮线性分析

对于 3 轮的线性分析，我们使用 MILP 方式进行线性活动 S 盒搜索，S 盒使用线性逼近表刻画，线性变换使用比特异或方式刻画。直接搜索 3 轮效率很低，我们使用一些技巧来搜索得到 3 轮最小线性活动 S 盒的下界和上界。

我们主要通过对 2 轮的详细线性活动 S 盒搜索得到 3 轮最小线性活动 S 盒的下界。具体的，通过搜索输入为  $\Gamma Z_i$ ，输出为  $\Gamma X_{i+1}$  的 2 轮最小线性活动 S 盒。

$$\Gamma Z_i \xrightarrow{\pi^{-1}} \Gamma Y_i \xrightarrow{\theta} \Gamma X_{i+1}$$

为了方便表示，我们将  $\Gamma Y_{ij}$  表示  $\Gamma Y_i$  的第  $j$  个面。我们按  $\Gamma Y_{ij}$  是否为 0，分为 32 种情况。然后对这 32 种情况分别求 2 轮的活动 S 盒个数，具体见表 4-4，其中 0 表示  $\Gamma Y_{ij}$  为 0，1 表示  $\Gamma Y_{ij}$  非 0。由于复杂度过高，当  $\Gamma Y_i$  中非 0 输入线性掩码面数大于 3 时，求得 2 轮最小线性活动 S 盒的个数不可行，利用定理 4-2，我们给出保守的最小活动 S 盒下界值。

表 4-4 2 轮线性活动 S 盒的详细值

| $\Gamma Y_{i,0}, \Gamma Y_{i,1}, \Gamma Y_{i,2},$<br>$\Gamma Y_{i,3}, \Gamma Y_{i,4}$ | 最小线性活动<br>S 盒数 | $\Gamma Y_{i,0}, \Gamma Y_{i,1}, \Gamma Y_{i,2},$<br>$\Gamma Y_{i,3}, \Gamma Y_{i,4}$ | 最小线性活动<br>S 盒数 |
|---------------------------------------------------------------------------------------|----------------|---------------------------------------------------------------------------------------|----------------|
| 0,0,0,0,0                                                                             | 0              | 0,0,1,1,1                                                                             | 23             |
| 0,0,0,0,1                                                                             | 12             | 0,1,0,1,1                                                                             | 24             |
| 0,0,0,1,0                                                                             | 12             | 0,1,1,0,1                                                                             | 23             |
| 0,0,1,0,0                                                                             | 12             | 0,1,1,1,0                                                                             | 23             |
| 0,1,0,0,0                                                                             | 12             | 1,0,0,1,1                                                                             | 23             |
| 1,0,0,0,0                                                                             | 12             | 1,0,1,0,1                                                                             | 23             |
| 0,0,0,1,1                                                                             | 18             | 1,0,1,1,0                                                                             | 23             |
| 0,0,1,0,1                                                                             | 18             | 1,1,0,0,1                                                                             | 23             |
| 1,0,0,1,0                                                                             | 18             | 1,1,0,1,0                                                                             | 23             |
| 0,1,0,0,1                                                                             | 18             | 1,1,1,0,0                                                                             | 24             |

|           |    |           |           |
|-----------|----|-----------|-----------|
| 0,1,0,1,0 | 18 | 0,1,1,1,1 | $\geq 24$ |
| 0,1,1,0,0 | 18 | 1,0,1,1,1 | $\geq 23$ |
| 1,0,0,0,1 | 18 | 1,1,0,1,1 | $\geq 24$ |
| 0,0,1,1,0 | 18 | 1,1,1,0,1 | $\geq 24$ |
| 1,0,1,0,0 | 18 | 1,1,1,1,0 | $\geq 24$ |
| 1,1,0,0,0 | 18 | 1,1,1,1,1 | $\geq 24$ |

从表 4-4 可以看出，当 $\Gamma Y_i$ 中非 0 输入线性掩码面数大于 2 时， $i$  轮和  $i+1$  轮连续 2 轮线性活动 S 盒的最小值为 23。

**定理 4-2:** 假设 $\Gamma Y_i$ 中原来有  $t$  个面输入线性掩码非 0，则再增加一个输入线性掩码非 0 的面后，2 轮的最小线性活动 S 盒数不会变小。

证明：假设 $\Gamma Y_i$  新增一个面的输入线性掩码后为 $\Gamma Y'_i$ ，令 $\Gamma Z_i = \pi(\Gamma Y_i)$ ， $\Gamma X_{i+1} = \theta(\Gamma Y_i)$ ； $\Gamma Z'_i = \pi(\Gamma Y'_i)$ ， $\Gamma X'_{i+1} = \theta(\Gamma Y'_i)$ 。由于 $\Gamma Y'_i$  比 $\Gamma Y_i$  新增一个面的输入线性掩码，所以根据  $\pi$  的性质， $\Gamma Z'_i$  是在 $\Gamma Z_i$  的基础上增加了非 0 的棱， $\Gamma X'_{i+1}$  是在 $\Gamma X_{i+1}$  的基础上增加了非 0 的棱，所以 $\Gamma Z'_i$  的最小线性活动 S 盒数不小于 $\Gamma Z_i$  的最小线性活动 S 盒数， $\Gamma X'_{i+1}$  的最小线性活动 S 盒数不小于 $\Gamma X_{i+1}$  的最小线性活动 S 盒数，即可得 2 轮的最小线性活动 S 盒数不会变小。

■

下面证明当第  $i$  轮小于 3 个面的输入掩码为非 0 时， $i$  轮、 $i+1$  轮和  $i+2$  轮连续 3 轮最小线性活动 S 盒数至少为 24。由于直接搜索时间复杂度过高，我们进行如下 2 轮搜索：

$$\Gamma Y_i \xrightarrow{\theta} \Gamma X_{i+1} \xrightarrow{\chi^{-1}} \Gamma Z_{i+1} \xrightarrow{\pi^{-1}} \Gamma Y_{i+1}$$

我们搜索输入为 $\Gamma Y_i$  输出为 $\Gamma Y_{i+1}$  的线性掩码对，为了得到精细的搜索结果，我们遍历搜索 $\Gamma Y_i$  和 $\Gamma Y_{i+1}$  面重量（我们称 $\Gamma Y_i$  中所有掩码非 0 的面的个数为 $\Gamma Y_i$  的面重量）为 1 和 2 的所有可能值，共  $15 \times 15 = 225$  种情况。搜索过程中，我们设定在下界值达到 23 时即停止搜索，搜索结果显示，有 224 种情况最小线性活动 S 盒不小于 23，只有 $\Gamma Y_{i,0} || \Gamma Y_{i,1} || \Gamma Y_{i,2} || \Gamma Y_{i,3} || \Gamma Y_{i,4} = 11000$ ， $\Gamma Y_{i+1,0} || \Gamma Y_{i+1,1} || \Gamma Y_{i+1,2} || \Gamma Y_{i+1,3} || \Gamma Y_{i+1,4} = 11000$  时，搜索得到的最小

线性活动 S 盒数为 20，此时 $\Gamma Y_i$ 的线性活动 S 盒数不小于 10，而 $\Gamma Y_{i+1}$ 的面重量为 2，由表 4-4 可知， $i+1$  和  $i+2$  轮的线性活动 S 盒数不小于 18，所以  $i$  轮、 $i+1$  轮和  $i+2$  轮 3 轮线性活动 S 盒数不小于  $10+18=28$ 。

由上述分析可知，当 $\Gamma Y_i$ 面重量不大于 2， $\Gamma Y_{i+1}$ 面重量不大于 2，则连续 3 轮线性活动 S 盒不小于 24。

综上，通过分析以下 3 种情况，可得置换 P 连续 3 轮最小线性活动 S 盒数不少于 24。

1) 当 $\Gamma Y_i$ 面重量大于 2 时，由表 4-4 可知， $i$  轮和  $i+1$  轮线性活动 S 盒不小于 23，可得连续 3 轮线性活动 S 盒不小于 24；

2) 当 $\Gamma Y_i$ 面重量不大于 2， $\Gamma Y_{i+1}$ 面重量大于 2，由表 4-4 可知， $i+1$  轮和  $i+2$  轮线性活动 S 盒不小于 23，可得连续 3 轮线性活动 S 盒不小于 24；

3) 当 $\Gamma Y_i$ 面重量不大于 2， $\Gamma Y_{i+1}$ 面重量不大于 2，则连续 3 轮线性活动 S 盒不小于 24。

事实上，对于 3 轮的线性活动 S 盒，上述 24 个活动 S 盒的计算非常保守，我们对于  $i$ 、 $i+1$  和  $i+2$  连续 3 轮的活动 S 盒进行 MILP 搜索，经过多种情况的尝试以及长时间的搜索，我们发现当 $\Gamma Y_i$ 只有 1 个面输入掩码非 0，且此面的输入掩码和输出掩码均为 6 时，得到 3 轮真实路径的最小线性活动 S 盒数为 64 个，即 3 轮最小线性活动 S 盒的上界为 64，具体见表 4-5。我们猜测 64 可能是 3 轮最小线性活动 S 盒数的下界。

表 4-5 3 轮线性活动 S 盒的详细值

| $\Gamma Y_{i,0}, \Gamma Y_{i,1}, \Gamma Y_{i,2}, \Gamma Y_{i,3}, \Gamma Y_{i,4}$ | 最小线性活动 S 盒数 | 每轮线性活动 S 盒个数 |
|----------------------------------------------------------------------------------|-------------|--------------|
| 1,0,0,0,0                                                                        | 64          | 6---6---52   |
| 0,1,0,0,0                                                                        | 64          | 6---6---52   |
| 0,0,1,0,0                                                                        | 67          | 6---6---54   |
| 0,0,0,1,0                                                                        | 68          | 6---6---56   |
| 0,0,0,0,1                                                                        | 66          | 6---6---54   |

综上所述，我们得到的置换 P 线性活动 S 盒数如表 4-6 所示。

表 4-6 置换 P 线性活动 S 盒数

| 轮数          | 1 | 2  | 3       |
|-------------|---|----|---------|
| 最小线性活动 S 盒数 | 1 | 12 | [24,64] |

#### 4.1.2.3. 积分分析

置换 P 整体结构为 SPN 结构，宽度为 1600 比特，使用 5 比特 S 盒，S 盒的代数次数为 3，因此，置换 P 是(5,3,320)-SPN。Todo[Tod15]给出了(5,2,320)-SPN 和(5,4,320)-SPN 的积分区分器结果，但没有给出(5,3,320)-SPN 的积分区分器结果，利用 Todo 在文献[Tod15]提出的算法 2，我们得到的置换 P（即(5,3,320)-SPN）积分区分器结果如表 4-7。

表 4-7 置换 P 的积分区分器

| 轮数    | 5        | 6         | 7         | 8          | 9          | 10         | 11         |
|-------|----------|-----------|-----------|------------|------------|------------|------------|
| 数据复杂度 | $2^{84}$ | $2^{245}$ | $2^{730}$ | $2^{1310}$ | $2^{1505}$ | $2^{1570}$ | $2^{1590}$ |

从表 4-7 可以看出，置换 P 的积分区分器的轮数为 11 轮，并且数据复杂度为  $2^{1590}$ ，若考虑实际的 Litchi 算法的安全性（ $2^{1024}$ ），区分器轮数只能到 7 轮。若对 S 盒和线性变换进行更详细的刻画，也许能得到更多轮数的积分区分器，但考虑置换 P 的轮数以及实际安全性，Litchi 算法是可以抵抗积分攻击的。

#### 4.1.2.4. 不可能差分分析

$$X_0 \xrightarrow{\chi} Y_0 \xrightarrow{\theta} Z_0 \xrightarrow{\pi} X_1 \xrightarrow{\chi} \dots X_{r-1} \xrightarrow{\chi} Y_{r-1} \xrightarrow{\theta} Z_{r-1} \xrightarrow{\pi} X_r$$

对于以上的一条  $r$  轮的差分路径，由于  $Y_i$  与  $X_{i+1}$  一一对应，为了搜索高效，我们只需找到一个  $X_0 \rightarrow Y_{r-1}$  的  $r$  轮不可能差分路径，就可以找到  $X_0 \rightarrow X_r$  的  $r$  轮不可能差分路径。

通过计算机搜索，我们找到了 4 轮的不可能差分路径，输入差分  $X_0=1||0^{1599}$ ，输出为  $Y_3=1||0^{1599}$  的 4 轮不可能差分路径，此时  $X_4$  的值如表 4-8 所示。

由于计算复杂度的原因，我们没有穷尽所有的差分路径，所以可能存

在更长轮数的不可能差分路径，但由于算法轮数有足够的冗余，我们相信置换  $P$  是可以抵抗不可能差分攻击的。

表 4-8 不可能差分路径  $X_4$  值

|                                                                                           |
|-------------------------------------------------------------------------------------------|
| 8000000140000000, 0000000000000000, 0000000000000000, 0000000000000000, 0000000000000000, |
| 0000000000000000, 0000000000000000, 0000020040000000, 0000000000000000, 0000000000000000, |
| 0000000000000000, 0000000000000000, 0000000000000000, 0000000000000000, 0000040040000000, |
| 0000000000000000, 0800000040000000, 0000000000000000, 0000000000000000, 0000000000000000, |
| 0000000000000000, 0000000000000000, 0000000000000000, 4000000040000000, 0000000000000000. |

#### 4.1.2.5. 零相关攻击

$$X_0 \xrightarrow{\chi^{-1}} Z_0 \xrightarrow{\pi^{-1}} Y_0 \xrightarrow{\theta} X_1 \xrightarrow{\chi^{-1}} \cdots X_{r-1} \xrightarrow{\chi^{-1}} Z_{r-1} \xrightarrow{\pi^{-1}} Y_{r-1} \xrightarrow{\theta} X_r$$

对于以上的一条  $r$  轮的线性路径，由于  $Y_i$  与  $X_{i+1}$  一一对应，为了搜索高效，我们只需找到一个  $X_0 \rightarrow Y_{r-1}$  的  $r$  轮零相关路径，就可以找到  $X_0 \rightarrow X_r$  的  $r$  轮零相关路径。

通过计算机搜索，我们找到了 4 轮的零相关路径， $X_0=1||0^{1599}$ ，输出为  $Y_3=1||0^{1599}$  的零相关路径，此时  $X_4$  值如表 4-9 所示。

表 4-9 零相关路径  $X_4$  值

|                                                                                           |
|-------------------------------------------------------------------------------------------|
| 8000000140000000, 4000000040000000, 0800000040000000, 0000040040000000, 0000020040000000, |
| 0000000000000000, 0000000000000000, 0000000000000000, 0000000000000000, 0000000000000000, |
| 0000000000000000, 0000000000000000, 0000000000000000, 0000000000000000, 0000000000000000, |
| 0000000000000000, 0000000000000000, 0000000000000000, 0000000000000000, 0000000000000000, |
| 0000000000000000, 0000000000000000, 0000000000000000, 0000000000000000, 0000000000000000. |

由于计算复杂度的原因，我们没有穷尽所有的零相关路径，所以可能存在更长轮数的零相关路径，但由于算法轮数有足够的冗余，我们相信置换  $P$  是可以抵抗零相关攻击的。

#### 4.1.2.6. 零和攻击

$S$  盒的低代数次数使得在置换上可构造内外零和区分器，不能将其视为理想随机置换。置换  $P$  轮函数  $f$  的代数次数是 3，经过  $R$  轮后代数次数的上界如表 4-10 所示（通过文献[BC13]的定理 2 进行次数估计）。

表 4-10 经  $R$  轮后代数次数的上界

| 轮数 $R$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|--------|---|---|---|---|---|---|---|---|
|--------|---|---|---|---|---|---|---|---|

|                |      |      |      |      |      |      |      |      |
|----------------|------|------|------|------|------|------|------|------|
| $\deg(f^R)$    | 3    | 9    | 27   | 81   | 243  | 729  | 1309 | 1503 |
| $\deg(f^{-R})$ | 3    | 9    | 27   | 81   | 243  | 729  | 1309 | 1503 |
| 轮数 R           | 9    | 10   | 11   | 12   | 13   | 14   | 15   | 16   |
| $\deg(f^R)$    | 1567 | 1589 | 1596 | 1598 | 1599 | 1599 | 1599 | 1599 |
| $\deg(f^{-R})$ | 1567 | 1589 | 1596 | 1598 | 1599 | 1599 | 1599 | 1599 |

全 16 轮置换可通过表 4-10 利用 7 轮逆向、1 轮中间轮、8 轮正向（或者 8 轮逆向、1 轮中间轮、7 轮正向，结果是一致的）进行零和区分，需要的数据复杂度为  $2^{1505}$ ，但目前尚未出现利用该性质对认证加密或哈希函数的实际攻击。

#### 4.1.2.7. 抗已知量子攻击

将 Litchi 函数看作随机函数，利用 Grover 算法可以得到经典（第二）原像攻击的平方加速，若输出长度为  $n$ ，量子（第二）原像攻击的复杂度为  $O(2^{n/2})$ 。对于碰撞攻击，利用 BHT 算法可以将经典生日攻击的复杂度  $O(2^{n/2})$  降低到  $O(2^{n/3})$ 。对于一些特殊结构，通过构造特定的周期函数利用 Simon 算法（结合 Grover 算法或 QAA）可以得到更好的攻击效果，但对于 Litchi 算法的上层结构，没有找到满足条件的周期函数。

若考虑置换  $P$  的具体结构，利用其差分、线性等特征结合 Grover 搜索等量子算法，可以对其进行量子差分、线性等攻击，将这些传统的分析方案量子化。但这些量子分析方法的只是将传统分析方案的搜索部分替换成 Grover 算法或 QAA 等量子算法，其攻击强度较对应的经典分析方案只能至多实现平方加速。由于 Litchi 算法的置换  $P$  在设计时已经充分考虑了抵抗经典差分、线性等攻击方法，在目前的分析下其对应的量子攻击方案的攻击复杂度高于通用量子攻击。

## 4.2 具体安全性

Litchi 算法根据其输出杂凑值长度可表示为 Litchi-512、Litchi-768 和 Litchi-1024，Litchi 算法的安全性结果如表 4-11：

表 4-11 Litchi 算法的安全性结果

| 算法          | 输出长度 | 经典计算条件          |                |                  | 量子计算条件           |                  |                       |
|-------------|------|-----------------|----------------|------------------|------------------|------------------|-----------------------|
|             |      | 原像              | 第二原像           | 碰撞               | 原像               | 第二原像             | 碰撞                    |
| Litchi -512 | 512  | 512             | 512            | 256              | 256              | 256              | $\geq 128$            |
| Litchi -768 | 768  | 768             | 768            | 384              | 384              | 384              | $\geq 192$            |
| Litchi-1024 | 1024 | 1024            | 1024           | 512              | 512              | 512              | $\geq 256$            |
| Litchi-XOF  | $d$  | $\min(d, 1024)$ | $\min(d, 960)$ | $\min(d/2, 512)$ | $\min(d/2, 512)$ | $\min(d/2, 480)$ | $\geq \min(d/4, 256)$ |

对于量子攻击，文献[GHJ+25]证明了当容量为  $c$ ，杂凑输出长度为  $|h|$  比特时，Litchi 的上层结构抗量子碰撞攻击复杂度不低于  $\min(O(2^{|h|/4}), O(2^{c/4}))$ ，而通用量子算法的抗碰撞攻击复杂度为  $O(2^{|h|/3})$ ，这并不表明通过我们的算法抗量子碰撞达不到通用量子抗碰撞复杂度，而是因为现在证明技术的原因，对上层结构的证明还不能得到最优的安全界，比如 Sponge 结构，现在可以证明得到的可抗量子碰撞攻击为不低于  $\min(O(2^{|h|/5}), O(2^{c/5}))$  [ACMT25]，这个界的安全性更低。事实上，现在没有找到任何优于通用量子抗碰撞复杂度的实际攻击。

## 4.3 其他安全特性

### 4.3.1 统计随机性测试

统计随机性测试主要检测摘要流的随机性，以 Litchi-1024 算法生成的摘要流进行测试。随机性检测的显著性水平设为  $\alpha=0.01$ 。检测项目包含频数检验、序偶检验、扑克检验、自相关检验和游程检验 5 项。消息样本生成方式包括随机消息生成方式、计数器生成方式、输出反馈生成方式、低汉明重量生成方式、高汉明重量生成方式 5 种，每种消息样本生成方式下产生  $2^{20}$  组 512 比特的消息样本，计算每组消息样本的消息摘要值并连接成摘要流。每种消息样本生成方式下最终得到  $2^{30}$  比特摘要流，将摘要流分为 1024 组，每组摘要流数据长度为  $2^{20}$  比特。在显著性水平为  $\alpha=0.01$  的设置下，对摘要流进行 5 项随机性检验，每种随机性检测项目的通过组数应不小于 1005 ( $\lceil 1024 \times (1 - \alpha - 3 \times (\alpha \times (1 - \alpha) / 1024)^{1/2}) \rceil = 1005$ )。从以下检测结

果可知，Litchi 算法能够通过随机性检测。

表 4-12 随机消息生成方式

| 检测序号 | 检测项目 | 检测参数 | 测组数  | 通过测试组数 |
|------|------|------|------|--------|
| 1    | 频率检验 | 无    | 1024 | 1016   |
| 2    | 序列检验 | 无    | 1024 | 1012   |
| 3    | 扑克检验 | 4    | 1024 | 1012   |
|      |      | 8    | 1024 | 1012   |
| 4    | 自相关  | 1    | 1024 | 1020   |
|      |      | 2    | 1024 | 1012   |
|      |      | 8    | 1024 | 1008   |
|      |      | 16   | 1024 | 1012   |
| 5    | 游程检验 | 无    | 1024 | 1020   |

表 4-13 计数器生成方式

| 检测序号 | 检测项目 | 检测参数 | 测组数  | 通过测试组数 |
|------|------|------|------|--------|
| 1    | 频率检验 | 无    | 1024 | 1011   |
| 2    | 序列检验 | 无    | 1024 | 1013   |
| 3    | 扑克检验 | 4    | 1024 | 1013   |
|      |      | 8    | 1024 | 1019   |
| 4    | 自相关  | 1    | 1024 | 1013   |
|      |      | 2    | 1024 | 1013   |
|      |      | 8    | 1024 | 1012   |
|      |      | 16   | 1024 | 1015   |
| 5    | 游程检验 | 无    | 1024 | 1013   |

表 4-14 输出反馈生成方式

| 检测序号 | 检测项目 | 检测参数 | 测组数  | 通过测试组数 |
|------|------|------|------|--------|
| 1    | 频率检验 | 无    | 1024 | 1011   |

|   |      |    |      |      |
|---|------|----|------|------|
| 2 | 序列检验 | 无  | 1024 | 1013 |
| 3 | 扑克检验 | 4  | 1024 | 1010 |
|   |      | 8  | 1024 | 1016 |
| 4 | 自相关  | 1  | 1024 | 1014 |
|   |      | 2  | 1024 | 1016 |
|   |      | 8  | 1024 | 1012 |
|   |      | 16 | 1024 | 1015 |
| 5 | 游程检验 | 无  | 1024 | 1014 |

表 4-15 低汉明重量生成方式

| 检测序号 | 检测项目 | 检测参数 | 测组数  | 通过测试组数 |
|------|------|------|------|--------|
| 1    | 频率检验 | 无    | 1024 | 1009   |
| 2    | 序列检验 | 无    | 1024 | 1013   |
| 3    | 扑克检验 | 4    | 1024 | 1014   |
|      |      | 8    | 1024 | 1010   |
| 4    | 自相关  | 1    | 1024 | 1012   |
|      |      | 2    | 1024 | 1012   |
|      |      | 8    | 1024 | 1009   |
|      |      | 16   | 1024 | 1013   |
| 5    | 游程检验 | 无    | 1024 | 1012   |

表 4-16 高汉明重量生成方式

| 检测序号 | 检测项目 | 检测参数 | 测组数  | 通过测试组数 |
|------|------|------|------|--------|
| 1    | 频率检验 | 无    | 1024 | 1019   |
| 2    | 序列检验 | 无    | 1024 | 1021   |
| 3    | 扑克检验 | 4    | 1024 | 1014   |
|      |      | 8    | 1024 | 1013   |
| 4    | 自相关  | 1    | 1024 | 1018   |

|   |      |    |      |      |
|---|------|----|------|------|
|   |      | 2  | 1024 | 1009 |
|   |      | 8  | 1024 | 1017 |
|   |      | 16 | 1024 | 1014 |
| 5 | 游程检验 | 无  | 1024 | 1018 |

### 4.3.2 抗长度扩展攻击安全性分析

长度扩展攻击的含义如下：对于一个未知的消息  $M$ ，已知其杂凑值  $H(M)$ ，是否存在某个比特串  $x$ ，可以计算出  $H(M||x)$ 。这个攻击方式对较早的基于 MD 结构的杂凑算法是有效的，因为它们往往给出了完整的最终链值，只要在填充过程上做出适当的调整，并按顺序计算压缩函数，即可成功构造出新的杂凑值。

对于 Litchi 算法使用的上层结构，由图 2-1 可以看出，杂凑输出阶段状态的下半部分异或了一个非零常数，所以当  $M||x$  填充后消息块数与  $M$  填充后消息块数不同时， $H(M)$  不可能为计算  $H(M||x)$  时的某个中间状态。若  $M||x$  填充后消息块数与  $M$  填充后消息块数相同时，若要利用  $H(M)$  计算  $H(M||x)$ ，需要对  $H(M)$  的最后输出阶段进行求逆，而这是不可行的。

综上，Litchi 算法是可以抵抗长度扩展攻击的。

## 5 性能评估

### 5.1 性能分析

#### 5.1.1 整体结构的设计效率

##### (1) 上层结构的软硬件兼容性

本算法采用的上层结构，将消息压缩与杂凑值输出统一为同一置换函数  $P$  的迭代，有效提升软硬件的实现效率。

在软件层面，采用循环迭代方式，便于实现为紧凑的循环代码，状态可连续存储于寄存器或内存中，减少函数调用开销。消息压缩与杂凑输出阶段的对称性降低了代码分支复杂度。在硬件层面，统一的置换函数意味着只需实例化一套  $P$  电路，通过反馈复用即可完成全部运算，资源利用率高。压缩阶段的异或操作与置换串行交替，控制逻辑简洁。

## （2）状态规模的选择

1600 比特状态是算法的核心参数，该规模处于现有杂凑算法典型值范围内，选取兼顾了安全强度与实现效率。对于软件实现效率，1600 比特可划分为 25 个 64 比特字或 50 个 32 比特字，适配主流处理器的寄存器宽度。在 64 位平台上，状态可完全载入通用寄存器，减少内存访问。对于硬件实现效率，1600 比特寄存器在 FPGA/ASIC 中面积可控。

### 5.1.2 核心组件的软硬件效率

#### （1）置换函数 P 的组合逻辑特性

置换函数 P 设计为纯组合逻辑，无内部状态存储。这一特性在软件与硬件层面均体现效率优势。

软件层面：纯组合逻辑反映为一系列按位运算（异或、移位、与、或等），无条件分支，利于编译器优化，且易于使用 SIMD 指令并行化。

硬件层面：组合逻辑电路无时钟依赖，单周期完成全部变换，关键路径长度仅由门级延迟决定，适合高频运行。

#### （2）S 盒比特切片实现的高效性

本算法采用的 5 比特 S 盒，其逻辑可直接转换比特切片实现，避免查表开销。比特切片实现时可将 1600 比特状态分解为 25 个 64 比特棱的运算，该设计在软件与硬件中均可快速实现。

## 5.2 性能测试

### 5.2.1 软件性能测试

#### （1）实现平台的详细配置说明

Litchi 杂凑算法在 64 位平台中测试评估，使用 Intel Core i3-6100 CPU，平台的具体要求如表 5-1 所示。

表 5-1 软件评估平台

| CPU                                       | 内存   | 操作系统    | 编程语言 | 编译器    | 指令集 | 密码库 |
|-------------------------------------------|------|---------|------|--------|-----|-----|
| Intel(R) Core(TM)<br>i3-6100 CPU @3.70GHz | 16GB | Win7-64 | 标准 C | VS2022 | 无   | 无   |

## （2）实现方法

①非线性变换 $\chi$ 的优化实现方法。根据方案描述，非线性变换 $\chi$ 需要 21 个运算操作。

$$B[0] = A[0] \& A[2] \& A[3] \oplus A[2] \& A[3] \& A[4] \oplus A[0] \& A[4] \oplus A[3] \& A[4] \oplus A[1];$$

$$B[1] = A[1] \& A[3] \& A[4] \oplus A[0] \& A[1] \oplus A[2];$$

$$B[2] = A[1] \& A[2] \oplus A[3];$$

$$B[3] = A[2] \& A[3] \oplus A[4];$$

$$B[4] = A[3] \& A[4] \oplus A[0]。$$

通过复用部分操作，降低运算操作数。

$$B[4] = A[3] \& A[4] \oplus A[0];$$

$$B[3] = A[2] \& A[3] \oplus A[4];$$

$$B[2] = A[1] \& A[2] \oplus A[3];$$

$$B[1] = B[4] \& A[1] \oplus A[2];$$

$$B[0] = B[4] \& B[3] \oplus A[1]。$$

可以看出，只需要 10 个运算操作，降低了一半以上的操作数。

②线性变换 $\theta$ 和换位变换 $\pi$ 的优化实现方法。将线性变换 $\theta$ 和换位变换 $\pi$ 结合在一起，从而减少运算操作数。

$$T[i][j] = A[i][j] \oplus (E_i \gg \gg t_{i,j}) \oplus F_i$$

$$B[i][j] = T[(i + 2j) \bmod 5][(3i + 3j) \bmod 5]$$

修改为：

$$B[(4i + 4j) \bmod 5][(i + 3j) \bmod 5] = A[i][j] \oplus (E_i \gg \gg t_{i,j}) \oplus F_i$$

③增加消息更新函数，提高代码的灵活性；

④将 4 类函数（Litchi-512、Litchi-768、Litchi-1024 和 Litchi-XOF）的底层消息压缩和杂凑输出阶段共用一个函数接口，减少代码冗余，同时便于管控代码；

⑤其他优化方法，将底层的小模块函数进行宏定义，降低代码调用路

径。

### （3）性能测试结果

表 5-2 Litchi-512/768/1024 与 SHA3-512 算法运算速度

| 级别 | 数据长度<br>(Bytes) | 算法运算速度(Mbps) |            |             |          |
|----|-----------------|--------------|------------|-------------|----------|
|    |                 | Litchi-512   | Litchi-768 | Litchi-1024 | SHA3-512 |
| S1 | 32              | 416.5        | 416.1      | 403.5       | 370.6    |
| S2 | 128             | 882.0        | 872.0      | 592.8       | 821.9    |
| S3 | 512             | 1439.1       | 1204.4     | 810.4       | 839.8    |
| S4 | 1024            | 1611.0       | 1313.7     | 862.8       | 905.7    |
| S5 | 4096            | 1767.8       | 1329.5     | 908.7       | 957.7    |
| S6 | 8192            | 1791.6       | 1362.6     | 910.9       | 959.9    |
| S7 | 16384           | 1812.2       | 1369.5     | 913.2       | 965.0    |
| S8 | 65536           | 1816.4       | 1374.2     | 921.8       | 968.1    |

#### 5.2.2 硬件性能测试

Litchi 杂凑算法输出杂凑值有 512 比特、768 比特和 1024 比特，对应 3 个硬件实现版本。3 个版本分别在 FPGA 和 ASIC 上进行了性能测试。

##### （1）实现平台的详细配置说明

在 xc7z100ffg900-2 FPGA 芯片和 SMIC 28nm 芯片上对 Litchi 杂凑算法进行实现、分析。使用 Modelsim SE 10.1c 仿真。FPGA 测试时综合工具使用 Vivado 2018.3，ASIC 测试时综合工具使用 Synopsys 公司的 Design Compiler 工具，版本为 syn\_v0\_2018.06-sp1。

##### （2）实现方法

硬件实现采用自顶向下的实现方法，算法顶层只有信号互连，模块间互相通过控制信号和数据信号进行通信。硬件电路逻辑框图如图 5-1 所示。

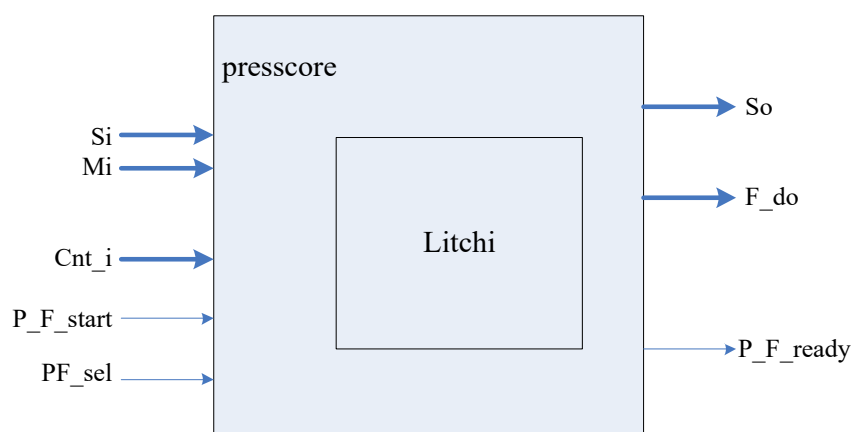


图 5-1 硬件电路逻辑框图

逻辑顶层信号定义如表 5-3 所示：

表 5-3 逻辑顶层信号定义表

| 信号名称             | 信号定义                   |
|------------------|------------------------|
| <i>Si</i>        | 迭代状态输入                 |
| <i>Mi</i>        | 消息分块输入                 |
| <i>Cnt_i</i>     | 计数器                    |
| <i>P_F_start</i> | 处理开始信号                 |
| <i>P_F_ready</i> | 处理完成信号                 |
| <i>PF_sel</i>    | 功能选择，0-消息压缩阶段，1-杂凑输出阶段 |
| <i>So</i>        | 迭代状态输出                 |
| <i>F_do</i>      | 杂凑结果输出                 |

### （3）性能测试结果

FPGA 测试结果如表 5-4 所示：

表 5-4 FPGA 测试结果

| 版本          | 资源占用       |      |      | 性能仿真结果       |                    |                    |
|-------------|------------|------|------|--------------|--------------------|--------------------|
|             | Flip Flops | LUTs | BRAM | 时钟周期<br>(ns) | 短包 64 字节<br>(Gbps) | 长包 1M 字节<br>(Gbps) |
| Litchi-512  | 1670       | 4393 | 0    | 2.9          | 10.27              | 20.55              |
| Litchi-768  | 1670       | 4352 | 0    | 2.9          | 10.27              | 15.41              |
| Litchi-1024 | 1670       | 4404 | 0    | 2.9          | 5.13               | 10.27              |

ASIC 测试结果如表 5-5 所示：

表 5-5 ASIC 测试结果

| 版本          | 资源占用 (GE) | 性能仿真结果    |                    |                    |
|-------------|-----------|-----------|--------------------|--------------------|
|             |           | 时钟周期 (ns) | 短包 64 字节<br>(Gbps) | 长包 1M 字节<br>(Gbps) |
| Litchi-512  | 41244     | 0.3       | 99.34              | 198.65             |
| Litchi-768  | 41446     | 0.3       | 99.34              | 149.00             |
| Litchi-1024 | 41697     | 0.3       | 49.67              | 99.33              |

## 6 特点声明

### 6.1 算法的创新性

#### (1) 创新设计安全高效上层结构

我们基于文献[GHJ+25]提出的  $\text{Sponge-F}^{\text{P, pd}}$  结构改进设计了 Litchi 杂凑算法的上层结构，相比广泛使用的  $\text{Sponge}$  结构，抗（第二）原像能力更强，吸收消息的效率更高。例如对于 Litchi-512，其上层结构相比 SHA3-512，吸收消息的效率可达到 2 倍。

#### (2) 创新设计简洁高安全的线性变换

我们设计了带参数的 320 比特的线性变换  $\text{Theta}$ ，其实现深度与 Keccak 的线性变换相同，但分支数由 4 提升到了 12，软件实现只需 18 个 64 位的异或和 10 个循环移位。基于不同的参数，可以设计出不同的线性变换，基于此线性变换，我们设计了安全简洁的 1600 比特内部置换。

#### (3) 创新设计安全简洁的内部置换

Litchi 算法内部使用 1600 比特置换，置换由 16 轮的轮数迭代而成，轮函数使用的部件都简洁高效，且所有运算都可以并行处理，比如非线性变换为 5 比特的 S 盒，比特切片实现时可将 1600 比特状态分解为 25 个 64 比特棱的并行处理；换位变换在硬件和软件方面都不带来任何实现代价。同时，内部置换各个部件之间有密切关联，达到很高的安全性，比如 3 轮达到全扩散，3 轮的差分活动 S 盒数很可能达到 60 左右，抗差分和线性等攻击能力相比 Keccak 算法置换的轮函数有大幅提升。

## 6.2 算法的简洁性

### (1) 上层结构简洁

与 Sponge 结构类似，上层结构基于内部置换 P，其它运算只有部分状态的前馈异或运算，无消息扩展等其它复杂运算。

### (2) 内部置换简洁

内部置换的所有运算部件运算只有与、异或和循环移位，并且每个部件的运算都非常简洁，运算次数和深度都很低。

## 6.3 算法的灵活性

### (1) 支持不同长度的杂凑输出

Litchi-512、Litchi-768 和 Litchi-1024 共用相同的内部置换 P，通过调整参数，即可以输出不同长度的杂凑输出。

### (2) 易于构造 XOF 和 MAC

与 Keccak 算法类似，Litchi 支持可扩展长度输出，并且通过密钥与消息的级联，即可实现 MAC 计算。

### (3) 优秀的并行性

算法的并行性主要体现在如下几个方面：

- ① 非线性变换支持并行实现；
- ② 线性变换支持并行实现；
- ③ 扩展输出 XOF 支持并行输出。

## 6.4 算法在不同平台上的实现性能优势

### (1) 高效的软硬件实现

简洁高效的密码部件 Litchi 算法所有运算均基于 64 比特设计，内部置换 P 的每个密码组件都简洁高效，单个 S 盒只需要 10 个基础运算，线性变换  $\theta_i$  只需要 18 个 64 位的异或和 10 个循环移位，换位变换不需要额外的实现代价。

### (2) 优秀的并行实现

上层结构在扩展输出 XOF 时支持并行输出，内部置换主要的密码组

件均支持并行输出。

## 7 与 SHA-3 算法比较

Litchi-512 算法与 SHA3-512 都是支持 512 比特的杂凑输出，二者内部置换状态都为 1600 比特。我们基于同样的平台和相同水平的优化实现技术对两个算法进行了软硬件实现，表 7-1 从安全性和效率方面给出了对两个算法的比较。

表 7-1 Litchi-512 与 SHA3-512 比较

| 算法         | 抗碰撞          | 抗原像          | 抗第二原像        | 不可分离性        | 软件效率       | 硬件效率      |
|------------|--------------|--------------|--------------|--------------|------------|-----------|
| Litchi-512 | $O(2^{256})$ | $O(2^{512})$ | $O(2^{512})$ | $O(2^{288})$ | 1816.4Mbps | 20.55Gbps |
| SHA3-512   | $O(2^{256})$ | $O(2^{512})$ | $O(2^{512})$ | $O(2^{512})$ | 968.1Mbps  | 8.94Gbps  |

注：软件平台为 Intel Core i3-6100 CPU，windows 操作系统。硬件效率为 FPGA 实现，综合工具使用 Vivado 2018.3，芯片为 xc7z100ffg900-2。

## 8 参考文献

- [ACMT25] Alagic, G., Carolan, J., Majenz, C., Tokat, S.: The Sponge Is Quantum Indifferentiable. pp. 2591-2630. IEEE Computer Society Press (2025).  
<https://doi.org/10.1109/FOCS63196.2025.00135>.
- [BC13] C.Boura and A.Canteaut, “On the influence of the algebraic degree of  $F^{-1}$  on the algebraic degree of GF, in IEEE Transactions on Information Theory, vol.59, no.1, pp.691-702, Jan.2013.
- [BDPV07] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. “Sponge functions”. Ecrypt Hash Workshop 2007. 2007. url: <http://sponge.noekeon.org/SpongeFunctions.pdf>.
- [BDPV08] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. “On the Indifferentiability of the Sponge Construction”. In: EUROCRYPT2008. Vol. 4965. LNCS. Springer, 2008, pp. 181-197.
- [BDPV11] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. “The Keccak Reference”. Submission to NIST (Round 3).

- [DEMS21] Christoph Dobraunig, Maria Eichlseder, Florian Mendel, and Martin Schl  ffer. Ascon v1.2: Lightweight authenticated encryption and hashing. *J. Cryptol.*, 34(3):33, 2021.
- [DM15] Dworkin, M.J., et al.: FIPS PUB 202-- SHA-3 standard: Permutation-based hash and extendable output functions (2015).
- [GHJ+25] Guo, C., Hu, K., Jiang, S., Fan, Y., Fu, Y., Preneel, B., and Wang, W.: Permutation-Based Hash from Non-Idealized Assumptions: Adding Feed-Forward to Sponge. *Cryptology ePrint Archive* (2025-1006).
- [HWL24] Lei H, Wang W, Zhang L. Symmetric twin column parity mixers and their applications[J]. *IACR Transactions on Symmetric Cryptology*, 2024, 2024(1).
- [Tod15] Yosuke Todo. "Structural Evaluation by Generalized Integral Property". In: *EUROCRYPT 2015*. Vol. 9056. LNCS. Springer, 2015, pp. 287-314.